SME309 final_report

郝宇鑫

谢毅

abdi

陈韬

Part1. Implementation of a five-stage pipeline ARM Processor

1. Introduction

In this project, we designed, implemented, and verified a **32-bit Five-Stage Pipelined ARM Processor**. Building upon the functional foundation of the single-cycle architecture developed in the previous lab, this design focuses on maximizing instruction throughput by overlapping the execution of multiple instructions. The processor partitions the execution flow into five distinct stages: **Instruction Fetch (IF)**, **Instruction Decode (ID)**, **Execute (EX)**, **Memory Access (MEM)**, and **Write Back (WB)**, separated by pipeline registers to maintain signal integrity across clock cycles.

While pipelining significantly improves performance, it introduces data and control hazards that threaten correctness. The core objective of this project was to implement robust hardware mechanisms to resolve these hazards without unnecessarily compromising performance. Our implementation features a dedicated **Hazard Unit** that coordinates three primary optimization techniques:

- **Data Forwarding (Bypassing):**To resolve Read-After-Write (RAW) hazards, the processor bypasses ALU results from the Memory or Writeback stages directly to the Execute stage. This minimizes pipeline stalls and ensures that dependent instructions receive the most up-to-date data immediately.
- **Hazard Detection and Stalling:**In cases where forwarding is impossible—specifically the **Load-Use Hazard**—the Hazard Unit injects "bubbles"

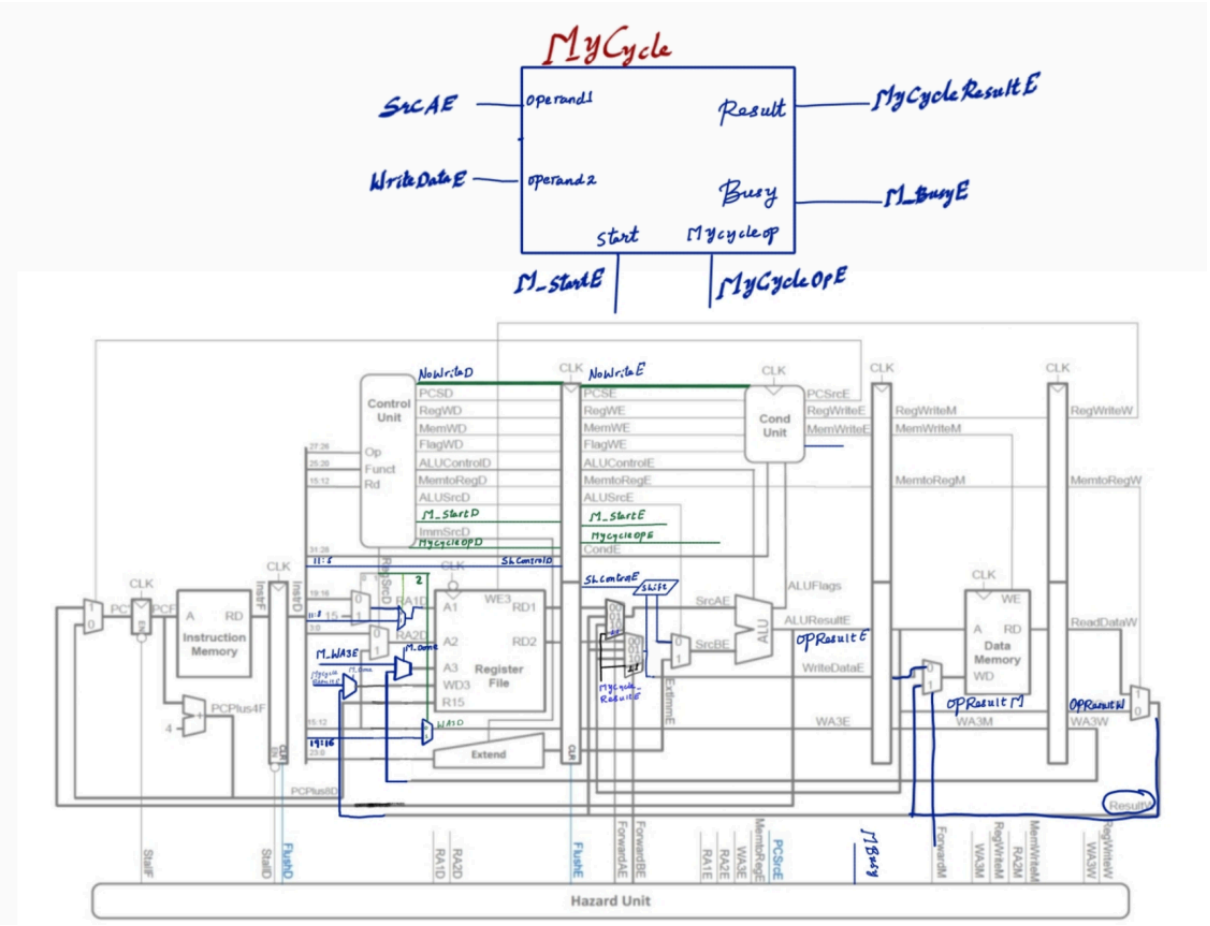
(NOPs) into the pipeline and freezes the Program Counter (PC) to preserve data integrity.

- **Control Hazard Optimization (Early BTA):**To reduce the penalty of branching, we implemented **Early Branch Target Acquisition (Early BTA)**. By moving the branch address calculation and decision logic from the Execute stage to the **Decode stage**, we reduced the branch penalty from two cycles to just one, significantly improving the efficiency of loops and conditional jumps.

Furthermore, a key requirement of this project was the integration of a **Parallel Multi-Cycle Unit**. Unlike standard blocking implementations, our design supports parallel execution. When a multiplication instruction is issued, it runs in the background, allowing the main pipeline to continue executing subsequent independent instructions. The Hazard Unit stalls the pipeline only when a specific data dependency on the multiplier result is detected, effectively hiding the latency of long arithmetic operations.

The correct operation of these features was verified through a comprehensive simulation testbench covering all hazard scenarios, followed by hardware synthesis and on-board verification.

2.Hardware Architecture



Schematic: the design of the processor is as follow

The processor architecture is organized into five distinct pipeline stages, each separated by clocked registers (IF/ID, ID/EX, EX/MEM, MEM/WB) to ensure signal stability and correct timing. Below is the description of the logic integrated into each stage.

2.1 Instruction Fetch (IF)

This stage is responsible for fetching the instruction from memory and determining the next program counter (PC) value.

- **Program Counter (PC):**The PC register holds the current address.
- **PC Priority Multiplexer:**Unlike a standard sequential fetch, the NextPC logic is governed by a priority multiplexer to handle control hazards:

1. *Highest Priority:*Late branch correction from the **Execute Stage** (Conditional Branches).

2. *Medium Priority:***Early BTA** target from the **Decode Stage** (Unconditional Branches).
3. *Lowest Priority:*Sequential execution (PC + 4).
    - **Instruction Memory:**A Harvard architecture setup where instructions are read based on the current PC.

2.2 Instruction Decode (ID)

This stage decodes the fetched instruction, reads operands, and computes branch targets early.

- **Decoder Unit:**The central control unit that parses the instruction bits to generate control signals (RegWrite, MemtoReg, ALUSrc, etc.) and detects multi-cycle operations (M_Start, MCycleOp).
- **Register File & Port Stealing Logic:**The register file provides two read ports (RD1, RD2) and one write port. To support **Parallel Execution**, the write port includes arbitration logic (Port Stealing). A multiplexer selects between the standard writeback data from the WB stage or the asynchronous result from the Multi-Cycle Unit when it completes.
- **Early BTA Adder:**To minimize control penalties, a dedicated adder computes the Branch Target Address (PC + Offset) in this stage rather than waiting for the Execute stage.
- **Extender:**Converts immediate values (8-bit or 12-bit) into 32-bit signed or unsigned integers.

2.3 Execute (EX)

This stage performs arithmetic, logic, and shift operations. Crucially, it supports parallel execution.

- **Forwarding Multiplexers:**Two 3-to-1 multiplexers (ForwardAE, ForwardBE) act as the entry point to the stage. Controlled by the Hazard Unit, they select inputs from either the Register File, the EX/MEM pipeline register, or the MEM/WB pipeline register to resolve **Read-After-Write (RAW)** hazards without stalling.
- **Barrel Shifter:**A combinational logic block placed before the ALU input, enabling single-cycle shift operations (LSL, LSR, ASR, ROR).

- **ALU (Arithmetic Logic Unit):**Performs the primary calculation (ADD, SUB, AND, ORR) and generates status flags (N, Z, C, V).
- **Parallel Multi-Cycle Unit (MCycle):**This module sits in parallel with the ALU. It receives M_Start to begin Multiplications/Divisions. It operates in the background, raising a Busy flag while the main pipeline continues executing subsequent independent instructions.

2.4 Memory Access (MEM)

This stage manages data exchange with the Data Memory.

- **Data Memory:**Stores and retrieves data based on the address calculated by the ALU.
- **Store Forwarding (ForwardM):**Specific logic detects **Memory-to-Memory copy** If a value being stored was loaded in the previous cycle, ForwardM bypasses the data directly to the memory write input, ensuring the STR instruction writes the correct value.

2.5 Write Back (WB)

The final stage where results are committed to the architectural state.

- **Result Selection:**A multiplexer selects the final data to be written back to the Register File, choosing between the ReadDataW (from Memory Load instructions) or OpresultW (from Arithmetic instructions).
- **Hazard Feedback:**The Register destination (WA3W) and Write Enable (RegWriteW) signals are fed back to the Hazard Unit to detect dependencies for instructions currently in the Memory or Execute stages. Additionally, the result is written to the Register File in the first half of the clock cycle, ensuring instructions in the **Decode stage** read the most current data.

3.Hazard Handling Techniques

Pipelining increases throughput but introduces dependencies between instructions. Our design resolves these issues through a dedicated **Hazard Unit** that controls stalling, flushing, and forwarding mechanisms. Furthermore, specific logic was implemented to satisfy the project requirements for Early BTA and Parallel Multi-Cycle execution.

3.1 Data Hazards

Data hazards occur when an instruction depends on the result of a previous instruction that has not yet been committed to the Register File.

- **Forwarding Mechanism:**
- To resolve Read-After-Write (RAW) hazards without pausing the pipeline, we implemented **Data Forwarding (Bypassing)**. The Hazard Unit monitors the source registers of the instruction in the **Execute**stage (RA1E, RA2E) and compares them with the destination registers of instructions in the **Memory** (WA3M) and **Writeback** (WA3W) stages.
- **EX Hazard:**If the data exists in the EX/MEM pipeline register, ForwardAE or ForwardBE is set to 10, selecting the result from the previous cycle (OpResultM) before it is written to memory or register.
- **MEM Hazard:**If the data exists in the MEM/WB pipeline register, the multiplexers select the result from the Writeback stage (ResultW), ensuring the most current data is used.
- **Store Forwarding:**We also implemented specific forwarding for the STR instruction (ForwardM), allowing a value loaded from memory in cycle *N* to be stored to memory in cycle *N*+1 without stalling.
    - **Load-Use Hazard:**
    - A special case arises when an instruction attempts to read a register immediately following a Load (LDR) instruction. Forwarding is impossible here because the data is physically located in Data Memory and has not yet been read into the processor.
- **Mechanism:**The Hazard Unit detects this sequence (Instruction in Decode reads a register being written by a Load in Execute).
- **Resolution:**The unit asserts StallF and StallD to freeze the Program Counter and the Decode stage for one cycle. Simultaneously, it asserts FlushE, inserting a **Bubble (NOP)** into the Execute stage. This delay allows the LDR to complete the memory access, after which the data can be forwarded normally.

3.2 Control Hazards

Control hazards arise from branch instructions, where the correct Program Counter (PC) is not known until the branch is resolved.

- **Early BTA (Branch Target Acquisition):**
- In a standard pipeline, branches are resolved in the Execute stage, incurring a 2-cycle penalty. To optimize this, we implemented **Early BTA**. We moved the branch target adder and unconditional branch detection logic (CondAL) into the **Decode (ID) Stage**. This allows the processor to determine the target address (BranchTargetD) and update the PC multiplexer immediately after decoding the instruction.
- **Pipeline Flushing:**
- Even with Early BTA, the instruction immediately following the branch (at *PC*+4) has already been fetched into the IF/ID register.
- **Mechanism:**When the Decoder detects a taken unconditional branch (BranchTakenD), it signals the Hazard Unit to assert FlushD.
- **Resolution:**This synchronously clears the IF/ID pipeline register, converting the erroneous fetched instruction into a NOP. This reduces the penalty for unconditional branches to just **1 cycle**. (Conditional branches resolved in Execute still incur a standard penalty/flush).

3.3 Parallel Multi-Cycle Handling

A key requirement was to allow the processor to continue executing independent instructions while the Multi-Cycle Unit (MCycle) performs long-latency operations (Multiply/Divide).

- **Scoreboarding Strategy:**
- Unlike blocking architectures, our Hazard Unit does **not**assert StallE simply because the Multiplier is busy. Instead, the instruction "fires" the multiplier and disappears from the main pipeline (becoming a "ghost" instruction).
- **Tracking:**The system tracks the destination register of the active multiplication in a scoreboard register (M_WA3E- the pending mul/div destination).
- **Dependency Check:**The Hazard Unit stalls the Decode stage (StallD) **only if** a subsequent instruction attempts to read M_WA3E. If the subsequent instructions are independent (e.g., do not read R13 during MUL R13...), they flow through the pipeline normally, achieving true parallel execution. This is tested by inserting an independent instruction between MUL instruction and instruction that is dependent on result of MUL unit. The processor runs parallel with Mcycle.

- **Write Port Arbitration (Structural Hazard):**
- A structural hazard occurs when the asynchronous Multiplier finishes and needs to write its result to the Register File, while the main pipeline attempts to write a different result in the same cycle (since there is only one Write Port).

- **Resolution:**We implemented a "Port Stealing" mechanism. When the MCycle unit completes, it generates a completion pulse. The system asserts a global **Structural Stall**, freezing all pipeline stages for exactly one cycle. During this cycle, the write port is muxed to accept the Multiplier result instead of the Writeback stage result, ensuring data integrity. At the same time, the data if forwarded to execute stage if the following instruction is in decode and is dependent.

4.Implementation Details

This section details the internal logic of the key modules developed to achieve the five-stage pipeline with parallel multi-cycle execution.

4.1 Hazard Unit (HazardUnit.v)

The Hazard Unit is the "brain" of the control flow, implemented as a purely combinational module. It accepts feedback from the Decode, Execute, Memory, and Writeback stages to generate three types of control signals:

- **Stall Logic:**The unit generates StallF and StallD to freeze the pipeline. Stalls are triggered by two specific conditions:

1. **Load-Use Hazard:**Detected when a specific instruction in Decode depends on a Register currently being loaded in Execute (Match_12D_E).
2. **Multi-Cycle Dependency:**To support parallelism, the unit does *not* stall blindly when the Multiplier is busy. Instead, it checks a **Scoreboard**: if the instruction in Decode explicitly reads the register reserved by the running Multiplier (PendingMulDest), it asserts StallD. To fix timing race conditions during the instruction issue cycle, we also included M_StartE in this check (MulStartHazard).

- *Safety Logic:*We implemented an input M_Busy_Delayed (registered) to extend the stall for one extra cycle, guaranteeing the Register File write completes before the dependent instruction is allowed to read.

- **Flush Logic:**FlushD and FlushE are generated to clear pipeline registers (insert bubbles) in response to control hazards or when a stall occurs in a previous stage (to prevent instruction duplication).
- **Forwarding Logic:**Combinational comparators detect if source operands in Execute match destination registers in Memory or Writeback stages to drive the ForwardAE, ForwardBE, and ForwardM

4.2 Decoder (Decoder.v)

The standard single-cycle decoder was enhanced to support Early BTA and Multi-Cycle control:

- **Early BTA Detection:**We implemented logic to identify unconditional branches (Opcode B, Condition AL) immediately. This generates the BranchTakenD signal, which is sent to the top-level PC Multiplexer and the Hazard Unit (to flush Fetch).
- **Control Separation:**The Decoder was updated to separate general PC updates (PCS) from Branch signals to prevent double-flushing in the Execute stage.
- **Multi-Cycle Signals:**Decodes specific opcodes for Multiply (MUL) to assert M_Start and set the MCycleOp mode, signaling the ARM core to initiate a background operation.

4.3 ARM Top-Level Module (ARM.v)

This module interconnects all stages and handles the structural changes required for parallel execution:

- **PC Priority Multiplexer:**To handle the Hybrid BTA strategy, we implemented a priority Mux for the Next PC:

1. *High Priority:*PCSrcE (Conditional Branch correction from Execute).
2. *Medium Priority:*BranchTakenD (Early Unconditional Branch from Decode).
3. *Low Priority:*PC_Plus_4F (Sequential flow).
    - **Structural Hazard Arbitration (Port Stealing):**Since the Register File has only one write port, we implemented arbitration logic. When the MCycle unit completes, it asserts a completion pulse. The ARM module triggers a **Structural Stall** (freezing all pipeline registers for 1

cycle) and switches the Register File's input mux to write the MCycleResult instead of the standard pipeline result.

- **Instruction Ghosting:**To allow the pipeline to continue while the multiplier runs, the RegWrite signal for the MUL instruction is masked (forced to 0) as it traverses the main pipeline. This prevents the MUL instruction from overwriting registers with invalid ALU data while the separate unit calculates.

4.4 Multi-Cycle Unit (MCycle.v)

- **Architecture:**This module uses a Finite State Machine (FSM) with IDLE and COMPUTING
- **Parallelism:**It accepts a Start pulse and immediately raises a Busy Crucially, the logic is decoupled from the main pipeline clock enables, allowing it to execute for 32 cycles in the background while the main pipeline continues processing independent instructions (StallE remains 0).

4.5 Register File (RegisterFile.v)

- **Clocking Strategy:**To resolve the structural hazard of decoding and writing in the same cycle, the write operation is triggered on the **falling edge (negedge CLK)**. This effectively splits the cycle: results are written in the first half, and the new data is available for reading by the Decode stage in the second half, ensuring instructions read the most up-to-date state.

5.Test Code and Simulation Waveforms

Test Bench Strategy:

The following assembly sequence was executed to verify all hazard conditions. The Wrapper.v was initialized with the instruction hex codes corresponding to this assembly.

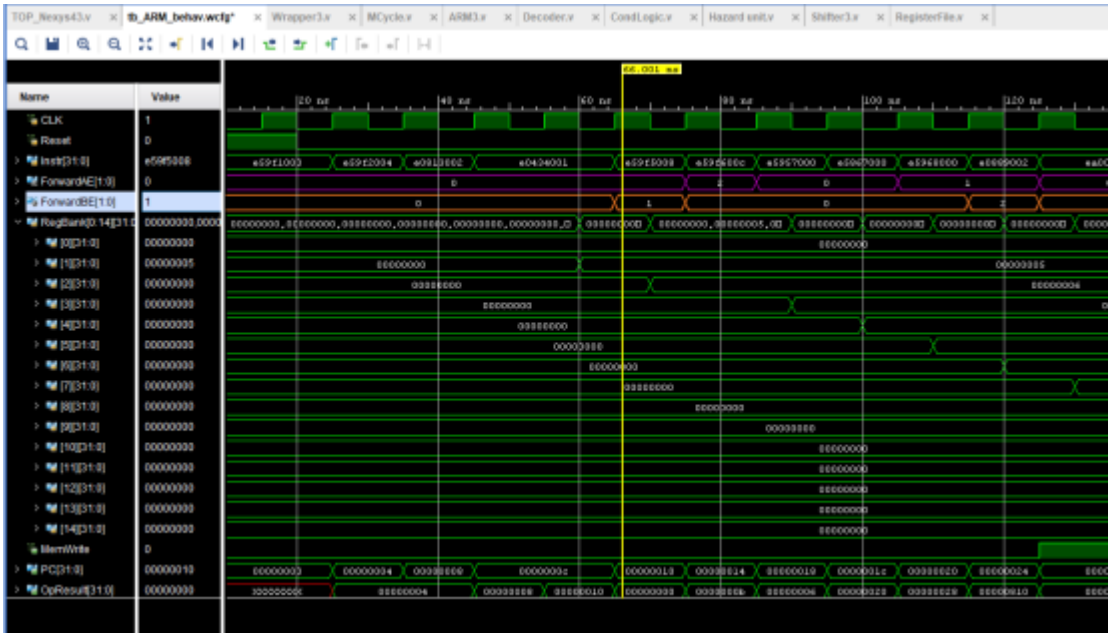
5.1 Read-After-Write (RAW) Hazard

- **Instructions:**

```
LDR   R1, constant1          ; R1 = 5
LDR   R2, constant2          ; R2 = 6
ADD   R3, R1, R2             ; R3 = 5 + 6 = 11 // R2 forwarded from W to E
```

SUB R4, R3, R1 ; R4 = 11 - 5 = 6 // R3 forwarded from M to E

- **Mechanism:**Forwarding from Writeback to Execute and Memory Stage to Execute Stage and .
- **Screenshot:**



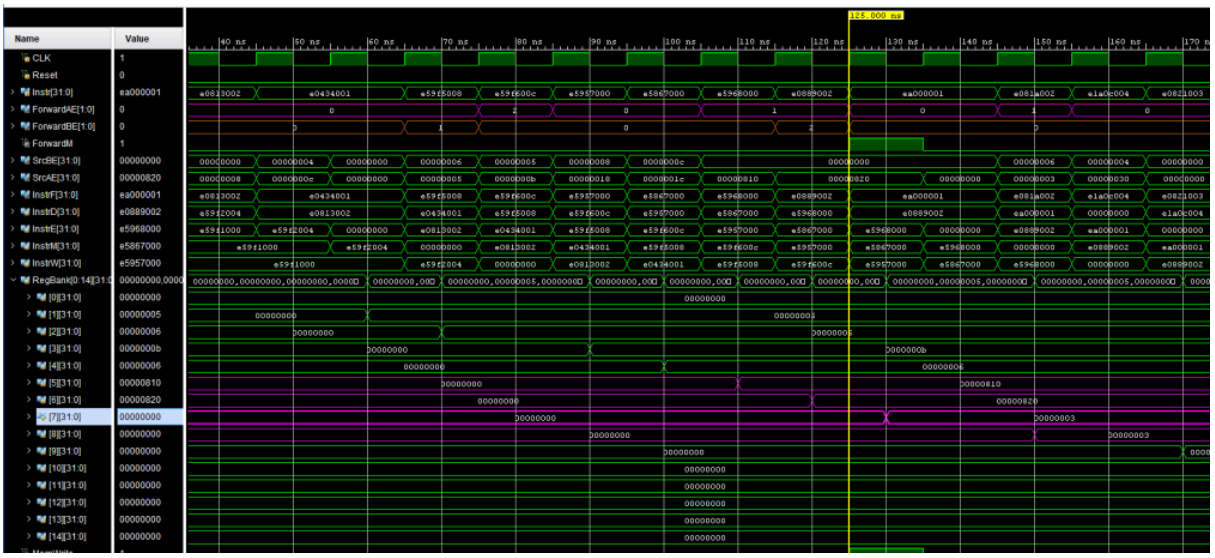
• **Observation:**The waveform shows ForwardAE and ForwardBE selecting the forwarded data, and SrcAE and SrcBE receives the correct value before writeback occurs.

5.2 Memory-to-Memory Copy

- **Instructions:**

```
LDR  R5, addr_src      ; R5 = 0x810
LDR  R6, addr_dst      ; R6 = 0x820
LDR  R7, [R5]          ; R7 = MEM[0x810] = 3   ; V=3
STR  R7, [R6]          ; MEM[0x820] = 3        ; V=3 copied
```

- **Mechanism:**Forwarding from Writeback Stage to Memory Stage.
- **Screenshot:**



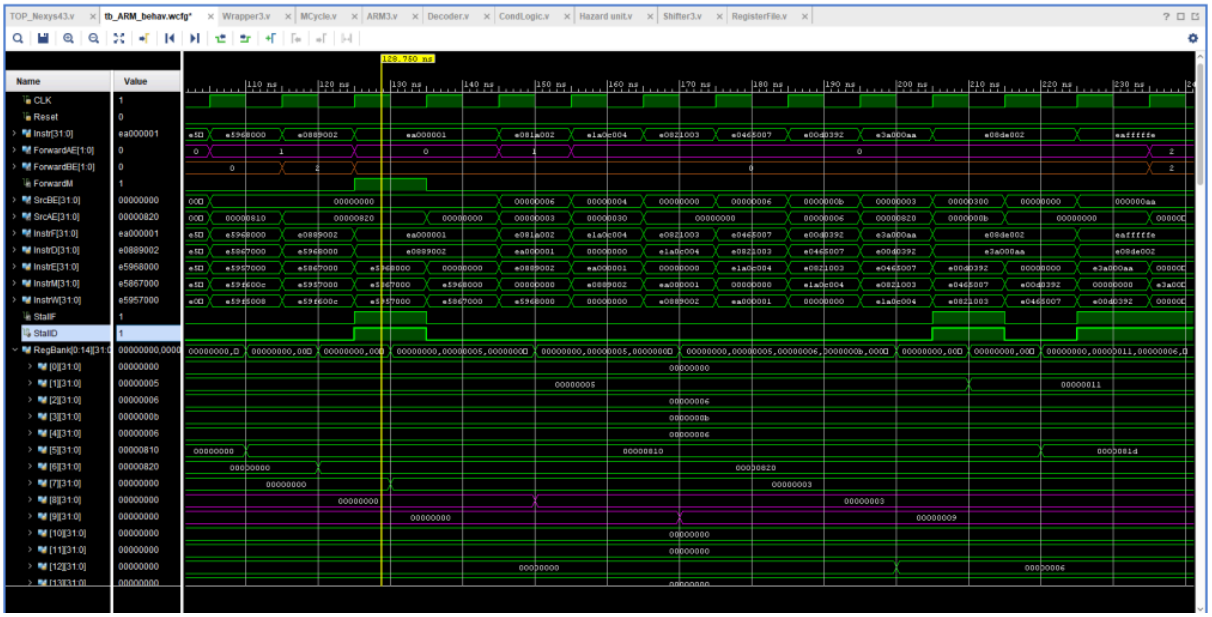
• **Observation:**ForwardM goes high, forwarding the loaded data directly to the memory write input.

5.3 Load-and-Use Hazard

- **Instructions:**

```
LDR   R8, [R6]              ; R8 = MEM[0x820] = 3   ; V=3
ADD   R9, R8, R2             ; R9 = 3 + 6 = 9
```

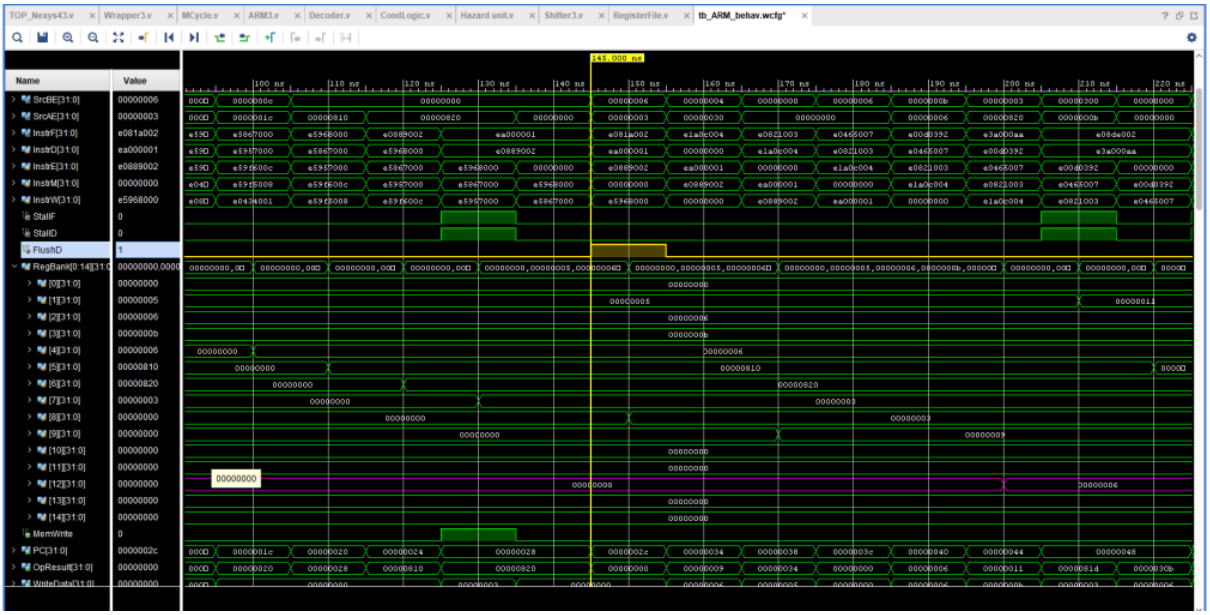
- **Mechanism:**Pipeline Stall (Bubble).
- **Observation:**StallF and StallD go high for 1 cycle; the ADD instruction waits in the Decode stage.
- **Screenshot:**



5.4 Control Hazards (Early BTA)

- **Instructions:**B skipfollowed by flushed instructions.
- **Mechanism:**Early Branch Decision + Pipeline Flush.
- **Observation:**FlushD goes high immediately in the Decode stage. The PC updates to the target address, skipping the NOP instructions efficiently (1 cycle penalty only).

Screenshot:

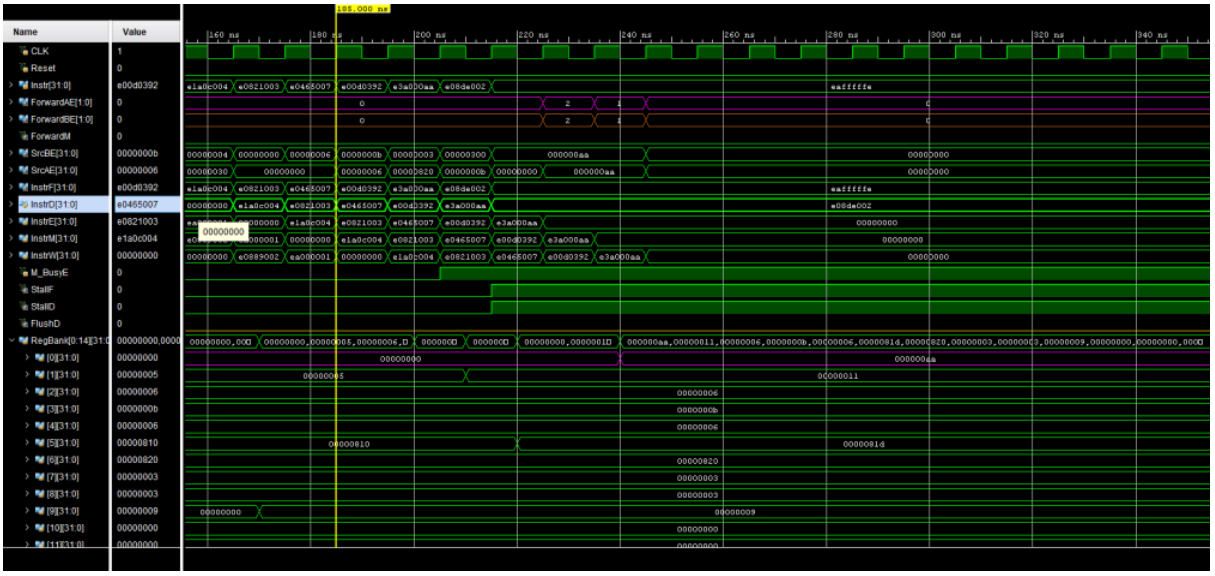


5.5 No Data Dependency (Parallel Execution)

- **Instructions:**

#Step 1: Start background operation
MUL R13, R2, R3; R13 = 6 * 11 = 66 (0x42);
Multiplier starts busy (Busy=1). RegWrite masked.
#Step 2: Parallelism Test (No Dependency)
#This instruction uses R0, totally independent of R13.
MOV R0, #0xAA; R0 = 0xAA (170);
STR R14, [R0, #800];
#Expectation: Executes IMMEDIATELY while MUL is busy.
#Step 3: Stall Test (With Dependency)
#This instruction uses R13, which is being calculated by MUL.
ADD R14, R13, R2; R14 = 66 + 6 = 72 (0x48);
#Expectation: STALLS in Decode until MUL finishes.

- **Mechanism:**Parallel Execution (Non-blocking) of MCycle if there is no dependency in the following instruction and stall if there is dependency.
- **Screenshot:**

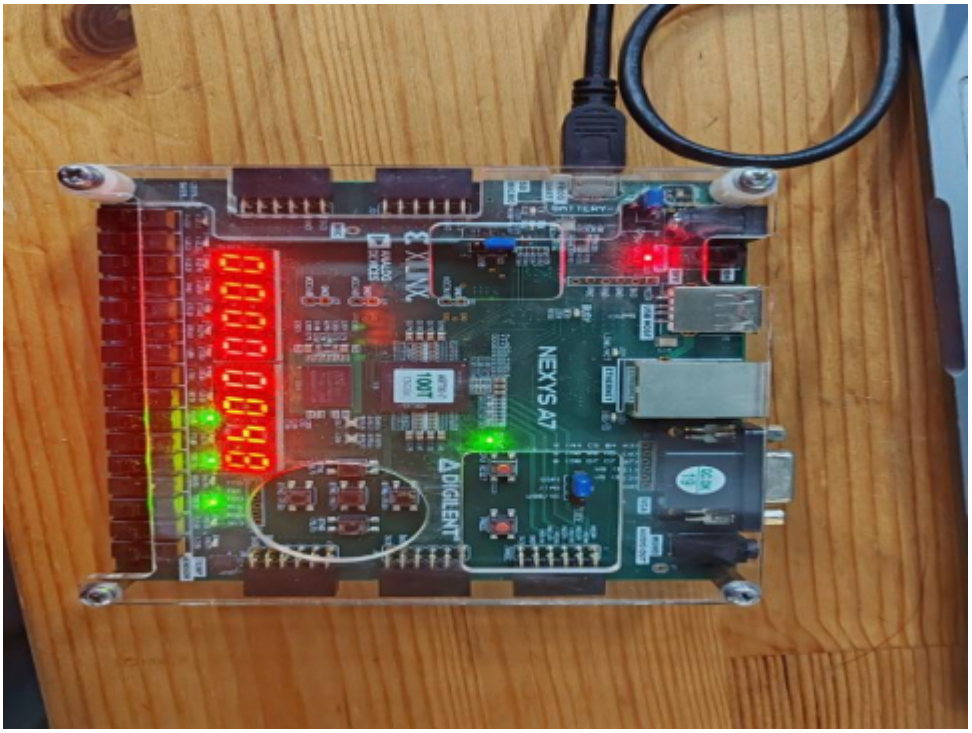


- **Observation:**M_Busy is high as soon as MUL enters Ex stage; the next instruction executes since there is no dependency with the Mcycle. The pipeline continues executing subsequent instructions while the multiplication runs in the background. However, since the ADD instruction

requires data from MCycle, it is stalled in decode stage with instruction following it until Mycycle is ready.

6.On Board Test Result

- **Setup:**Brief description of the FPGA board used and the clock frequency setup.
- **Result:**[Insert a clear photo of the FPGA board LEDs/Seven-Segment display showing the final PC value or Result register].



- **Analysis:**Confirm that the output matches the expected result from the simulation.

Appendix

- **Source Code:**(Optional, or provide a link to the Git repository).

Instruction Memory Hex: The hex codes used for the test bench.

Part2. Design and Benchmarking of a RISC-V Processor

1.Overview of the RISC-V ISA, and comparison with the ARM architecture

1.1 Overview of the RISC-V Instruction Set Architecture

RISC-V, an open-source Reduced Instruction Set Computer (RISC) architecture, was first proposed by the University of California, Berkeley. It aims to provide a concise, scalable, and patent-free instruction set standard for academic research, education, and industrial applications. Unlike traditional closed ISA, RISC-V's defining feature is its open and modular design philosophy, enabling processor designers to flexibly tailor or expand instruction set functionalities according to specific application requirements. This project implements and validates RV32I, the 32-bit basic integer instruction set of RISC-V, which represents the most fundamental and compact implementation in the RISC-V architecture. RV32I features the following key characteristics:

- **Programmer's Model:**

32 general-purpose integer registers (x0–x31), each with a 32-bit width;

x0 is fixed to the constant 0 by hardware to simplify instruction encoding and hardware design;

single program counter (SPC), byte addressing

- **Load/Store architecture:**RISC-V employs a rigorous Load/Store architecture design:

Only Load (LW, LH, LB, etc.) and Store (SW, SH, SB, etc.) instructions can access the data memory.

Arithmetic and logical operations can only be performed between registers.

The design significantly simplifies the pipeline execution and data path control, which is conducive to efficient implementation and formal verification.

- **Fixed Length and Regularized Coding:**In RV32I:

All basic instructions are 32-bit fixed-length instructions.

The instruction format is highly standardized, primarily comprising R-type, I-type, S-type, B-type, U-type, and J-type.

The rule-based encoding reduces the complexity of decoding logic and enables the instruction decoding to be completed quickly and in parallel, which is very suitable for the five-stage pipeline structure.

- **Modularity and Scalability:**RISC-V adopts a design approach combining a basic instruction set with optional extensions:

I: Basic Integer Instruction Set

M: Multiplication and Division Extension

F/D: floating point extension

C: Compression instruction extension

This modular design enables flexible integration of new features without compromising existing ISA compatibility, which is a key factor behind RISC-V's rapid development.

- **Simplicity and Implementability:**The RISC-V instruction set deliberately avoids complex and redundant semantics, embodying the design philosophy of "simple instructions + efficient microarchitecture" This characteristic makes it an ideal CPU design target for course projects, enabling students to focus on critical aspects like pipelines, risk-based processing, and performance optimization.

1.2 Comparison between RISC-V and ARM Architecture

The ARM architecture is currently the most widely adopted commercial instruction set architecture (ISA) in embedded systems and mobile devices, while RISC-V is an emerging open ISA that has gained rapid traction in recent years. These two architectures differ significantly in their design objectives and implementation philosophies.

- **Authorization and Ecosystem Model:**

RISC-V: Fully open source, free to implement by any organization or individual without licensing fees

ARM: Operating under a commercial licensing model, chip manufacturers must pay IP licensing fees to ARM. As a result, RISC-V demonstrates clear advantages in academic research and custom processor design, while ARM enjoys a more mature commercial ecosystem in the industrial sector.

- **Instruction set design concept:**

The RISC-V architecture emphasizes:

Minimalism;

Highly orthogonal and uniformly regular;

Optimized for hardware implementation

While the ARM architecture has undergone a long-term evolution:

Supports multiple earlier instruction sets including ARM, Thumb, and Thumb-2;

ISA is highly complex;

Extensive industrial-level optimization has been implemented for high performance and low power consumption.

- **Instruction Coding and Hardware Complexity:**

Architecture	**RISC-V**	**ARM**
Instruction length	Fixed 32-bit (RV32I)	16/32 bit hybrid
Coding rule	Uniform rule	Multiple coding methods
Decoding complexity	Low	Higher
Teaching friendliness	Very high	Normal

- **Architectural philosophy difference:**

RISC-V： Open, clean, and scalable, with performance optimization achieved through micro-architecture

ARM： "Industrial-grade mature architecture" emphasizes energy efficiency, system integration, and commercial viability

- **Load/Store architecture:**RISC-V employs a rigorous Load/Store architecture design:

Only Load (LW, LH, LB, etc.) and Store (SW, SH, SB, etc.) instructions can access the data memory.

Arithmetic and logical operations can only be performed between registers.

The design significantly simplifies the pipeline execution and data path control, which is conducive to efficient implementation and formal verification.

2.Description of CPU Modules and Implementation

This project implements a five-stage pipelined RISC-V CPU supporting the RV32I instruction set. While maintaining single-cycle CPU functionality, the

processor introduces a pipeline mechanism to enhance instruction throughput. The five pipeline stages are:

Instruction Fetch (IF) → Instruction Decode (ID) → Execute (EX) → Memory Access (MEM) → Write Back (WB)

2.1 Instruction Fetch (IF) Stage

The IF stage retrieves the next executable instruction from the instruction memory and calculates its address.

- **Program counter (PC) register**
- **Instruction Memory**
- **Adder (PC + 4)**
- **IF/IDpipeline register**

Under normal conditions, the PC increments by 4 per clock cycle.

If a branch or jump occurs, the PC is updated to the branch target address.

The retrieved instruction is written to the IF/ID register along with the current PC value.

2.2 Instruction Decode / Register Fetch (ID) Stage

The ID stage completes instruction decoding, register reading and control signal generation.

- **Instruction decoder**
- **Register File**
- **Immediate Generator**
- **Control Unit**

Read both rs1 and rs2 from the register stack simultaneously

Generate the corresponding immediate value based on the instruction type

Generate control signals such as ALUOp, MemRead, MemWrite, and RegWrite.

The decoding result is stored in the ID/EX pipeline register.

2.3 Execute (EX) Stage

The EX stage performs arithmetic and logical operations, including branch judgments and address calculations.

- **ALU**

- **ALU control unit**
- **Branching comparison logic**
- **Forwarding Unit**

ALU supports addition, subtraction, logical operations, comparison, and shift operations.

For branch instructions, determine whether the condition is met at this stage.

Calculate the valid address for Load/Store Addressing data-related RAW risks through the pre-processor mechanism

2.4 Memory Access (MEM) Stage

The MEM stage handles interactions with the data memory.

- **Data Memory**
- **EX/MEM, MEM/WB pipeline registers**
- **Immediate Generator**

The load instruction reads data from memory.

The store instruction writes data to memory.

Non access memory instruction directly transfers ALU results

2.5 Write Back (WB) Stage

The WB stage writes the final result back to the register stack.

- **Multiplexer (selects ALU results or memory data)**
- **Register write back control logic**

Select the data source for writeback based on the control signal

Write the result to the target register on the clock rising edge

2.6 Pipeline Registers and Hazard Handling

To ensure the pipeline's proper operation, the design implements multiple hazard mitigation mechanisms:

- **Pipeline register :**IF/ID ID/EX EX/MEM MEM/WB
- **Data hazardmanagement:**

Data Forwarding

Load-Use exploits are mitigated by inserting Stall or NOP instructions

- **Control hazard:**

Flush pipeline when a branch error occurs

Ensure correct execution semantics

2.7 Multi-cycle Units

In the extended design, multiplication or division can be implemented as independent multi-cycle modules.

The module can run in parallel with the pipeline

If data dependencies exist, Stall ensures correctness

1.3 Detailed Description of RISC-V Submodules

After the comparison, let's explain the functions and implementation methods of each part specifically based on the RISC-V architecture.

1.3.1 Program Counter

In the specific code writing, we divide this part into PC update and PC MUX. We still use asynchronous updates for the update method. For the MUX, based on the code analysis, we have three writing possibilities: `PC+4`, `PC+imm`, `x+offset`. Among them, the special value `PC+imm` is the output requirement for `BNE`, `BEQ`, and `JAL` instructions. The first two instructions also need the result of comparing `rs1` and `rs2` (`zero`), so we set up an OR gate to connect these three conditions as the control signal for the MUX. Another special value `x+offset` is controlled by the `JALR` instruction without other preconditions, so we directly introduce this control signal from the Control Unit to the MUX.

```
module ProgramCounter(
    input CLK,
    input Reset,
    input flush,
    input pause,
    input [31:0] NextPC,

    output reg [31:0] PC,
    output reg [31:0] if_pc
```

```
    );
always @(posedge CLK) begin
        if(Reset) begin
            PC <= 0;
            if_pc <= 0;
        end else if(flush) begin
            PC <= NextPC;//并非冗余，有个优先级的关系
            if_pc <= NextPC;
        end else if(pause) begin
            // 空操作
            // 阻止寄存器值改变
        end else begin
            PC <= NextPC;
            if_pc<= NextPC;
        end
    end
endmodule
```

1.3.2 Control Unit

We previously listed the decoder for the Control Unit in Figure 1.3, which is our core part. We can implement most decoding functions by selecting cases based on the `op` in the instruction:

```
always @(*) begin
    case(op)
        7'b0010011:{MemtoReg,MemWrite,ALUSrc,RegWrite,PC4,set,xset,PCimm}=8'b0_0_1_1_0_0_0_0;//addi & shift
        7'b0110011:{MemtoReg,MemWrite,ALUSrc,RegWrite,PC4,set,xset,PCimm}=8'b0_0_0_1_0_0_0_0;//R-type
        7'b0000011:{MemtoReg,MemWrite,ALUSrc,RegWrite,PC4,set,xset,PCimm}=8'b1_0_1_1_0_0_0_0;//lw
        7'b0100011:{MemtoReg,MemWrite,ALUSrc,RegWrite,PC4,set,xset,PCimm}=8'bX_1_1_0_0_0_0_0;//sw
        7'b1100011:{MemtoReg,MemWrite,ALUSrc,RegWrite,PC4,set,xset,PCimm}=8'b0_0_0_0_0_0_0_0;//beq/bne
        7'b1101111:{MemtoReg,MemWrite,ALUSrc,RegWrite,PC4,set,xset,PCim
```

```
m}=8'b0_0_0_1_1_1_0_0;//jal
        7'b1100111:{MemtoReg,MemWrite,ALUSrc,RegWrite,PC4,set,xset,PCim
m}=8'b0_0_1_1_1_0_1_0;//jalr
        7'b0010111:{MemtoReg,MemWrite,ALUSrc,RegWrite,PC4,set,xset,PCim
m}=8'b0_0_0_1_0_0_0_1;//auipc
        default:{MemtoReg,MemWrite,ALUSrc,RegWrite,PC4,set,xset,PCimm}
=8'b0_0_0_0_0_0_0_0;
    endcase
end
```

However, we have not yet generated control signals for the ALU. Here we also need to rewrite the relevant truth table:
We can use case selection to implement this, but considering that we not only need to judge `op` but also `funct3` and `funct7`, we use `if-else` in the code to implement it. In addition, the function definition of `ALUControl` can also be learned from this table.

```
always @(*) begin
    condition_branch = 0;
    out = 32'b0;
    case (aluc)
        5'b00000: out = a + b;
        5'b00001: out = a - b;
        5'b00010: out = a & b;
        5'b00011: out = a | b;
        5'b00100: out = a ^ b;
        5'b00101: out = a << b;
        5'b00110: out = ($signed(a) < ($signed(b))) ? 32'b1 : 32'b0;
        5'b00111: out = (a < b) ? 32'b1 : 32'b0;
        5'b01000: out = a >> b;
        5'b01001: out = ($signed(a)) >>> b;
        5'b01010: begin
            out = a + b;
            out[0] = 1'b0;
        end
        5'b01011: condition_branch = (a == b) ? 1'b1 : 1'b0;
        5'b01100: condition_branch = (a != b) ? 1'b1 : 1'b0;
        5'b01101: condition_branch = ($signed(a) < $signed(b)) ? 1'b1 : 1'b0;
        5'b01110: condition_branch = ($signed(a) >= $signed(b)) ? 1'b1 : 1'b0;
        5'b01111: condition_branch = (a < b) ? 1'b1: 1'b0;
        5'b10000: condition_branch = (a >= b) ? 1'b1: 1'b0;
        default: out = 32'b0;
    endcase
end
```

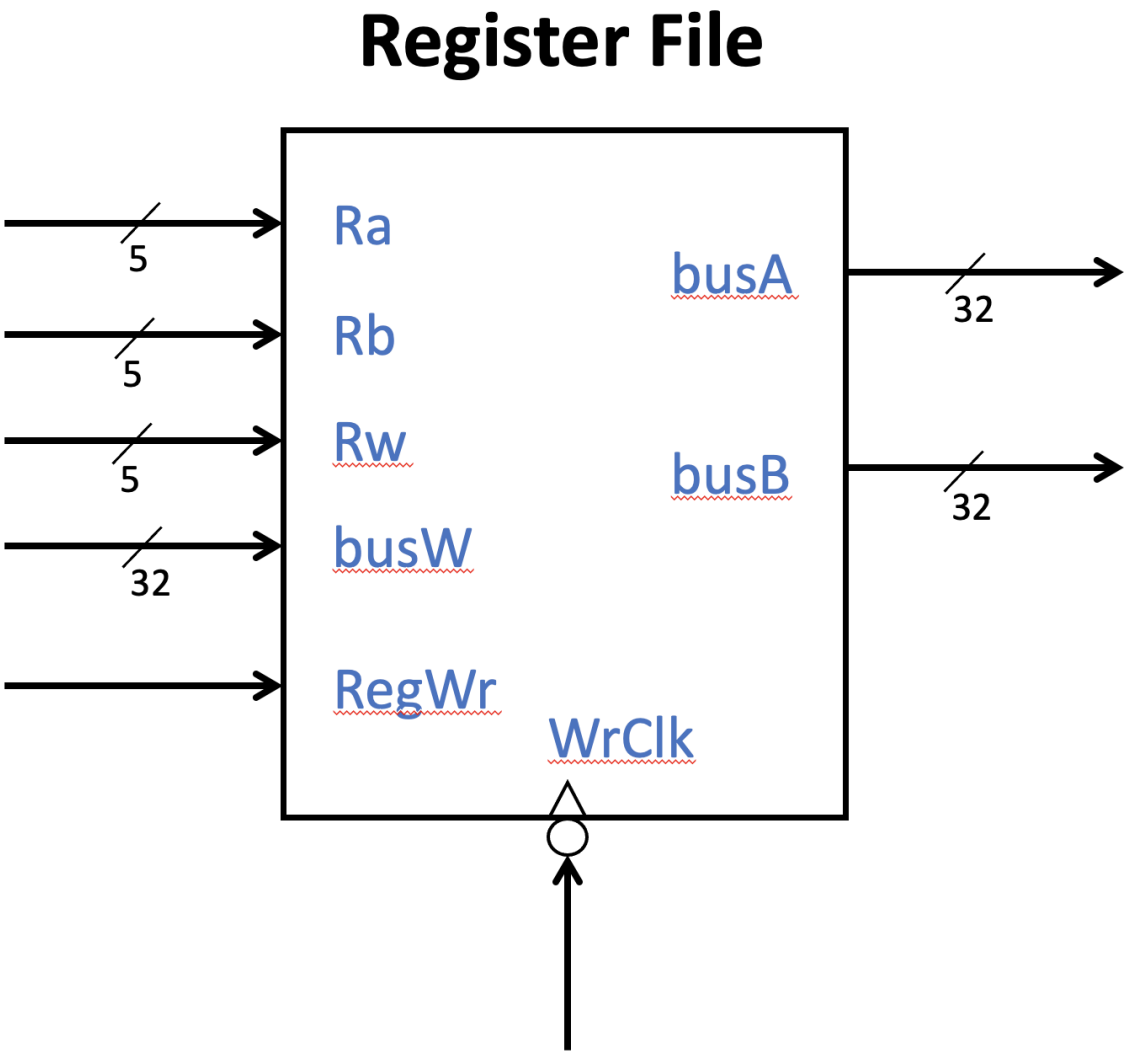
1.3.3 Register File

We adopted the writing style from the ARM architecture for this part as a whole. However, due to the requirement that `Reg[0]` must always be 0, we added extra code for the write part:

```
module RegisterFile(
    input CLK,
    input WE3,
    input [4:0] A1,
    input [4:0] A2,
    input [4:0] A3,
    input [31:0] WD3,

    output reg [31:0] RD1,
    output reg [31:0] RD2
);
```

```
reg [31:0] RegBank[0:31];
reg [4:0] addr1_reg, addr2_reg;
integer i;

initial begin
    for (i = 0; i < 32; i = i + 1)
        RegBank[i] = 32'b0;
    RD1 = 32'b0;
    RD2 = 32'b0;
end

// 上升沿写入
always @(posedge CLK) begin
    if(WE3 && A3 != 0) RegBank[A3] <= WD3;
end

// 组合逻辑读取 - RD1
always @(*) begin
    if(A1 == 5'h0) begin
        // x0寄存器硬连线到0
        RD1 = 32'h0000_0000;
    end else begin
        if(WE3 && (A1 == A3) && (A3 != 0)) begin
            // 数据前递：如果正在写入的寄存器是当前要读取的
            RD1 = WD3;  // 使用要写入的新值
        end else begin
            // 正常读取寄存器值
            RD1 = RegBank[A1];
        end
    end
end

// 组合逻辑读取 - RD2
always @(*) begin
    if(A2 == 5'h0) begin
        // x0寄存器硬连线到0
        RD2 = 32'h0000_0000;
```

```
        end else begin
            if(WE3 && (A2 == A3) && (A3 != 0)) begin
                // 数据前递：如果正在写入的寄存器是当前要读取的
                RD2 = WD3;  // 使用要写入的新值
            end else begin
                // 正常读取寄存器值
                RD2 = RegBank[A2];
            end
        end
    end
endmodule
```



1.3.4 Immediate Data

As we learned from the previous comparison, we parse the code directly here instead of relying on control signals from the Control Unit. Based on our experience in writing the Control Unit, we directly use `case` to categorize each instruction and its corresponding parsing based on `op`. In addition, since each instruction needs to determine the sign of the immediate value and fill in the bits, we set up `if-else` separately under various conditions of the `case` to categorize them:

```
7'b0000011: begin//LW
   if(instr[31] == 1) imm = {20'hfffff,instr[31:20]};
   else imm = {20'h0,instr[31:20]};
end
```


1.3.5 Shifter

We use the same implementation method as in the ARM architecture, using a five-level multiplexer (corresponding to five bits) for displacement. Considering the simplicity of RISC-V instructions, to prevent incorrect judgment, we directly introduce instructions to parse and determine the displacement method.

1.3.6 ALU

In RISC-V we only have five operations: Add, Subtract, AND, OR, XOR. We calculate directly using corresponding symbols.

```
3'b000: ALUResult = Src1 + Src2;
3'b001: ALUResult = Src1 - Src2;
3'b010: ALUResult = Src1 & Src2;
3'b011: ALUResult = Src1 | Src2;
3'b100: ALUResult = Src1 ^ Src2;
```

Based on the previous comparison, we introduced a new output quantity `zero` to determine whether the preconditions for `BNE` or `BEQ` instructions are met: `assign zero = (ALUResult==32'b0) ? 1'b1: 1'b0;`

1.3.7 Result MUX

Because J-type instructions added new ways of writing for our input (such as `PC+4`, `PC+imm<<12`), there are four ways in total. We realize the selection of writing methods through `if-else` syntax, which is represented as 3 MUXs in the architecture diagram.
The code for this part is as follows:

```
always @(*) begin
    if (MemtoReg) begin
        Result = ReadData;
    end
    else if (PC4) begin
        Result = PC_plus_4;
    end
    else if (PCimm) begin
        Result = PC_imm;
    end
    else begin
        Result = ALUResult;
    end
end
```

![Image Placeholder: Picture 8.11: Result MUX Part in RISC-V Architecture]

1.3.8 Else

We used the Wrapper and Top written for the ARM architecture this semester. Therefore, regarding Instruction Memory, we will display it in the test code later, and the writing of the Data Memory module will not be explained. The rest only requires relevant wiring to connect the modules. Overall, the connection is simpler than the ARM architecture.

1.4 Test Code Display and Simulation Waveform Analysis

The following is the code for the execution instructions we wrote manually for testing:

```
INSTR_MEM[0] = 32'h0010_0093; //addi x1=x0+1=0+1=1
INSTR_MEM[1] = 32'h0020_0113; //addi x2=x0+2=0+2=2
```

```
INSTR_MEM[2] = 32'h0011_01B3; //add x3=x2+x1=2+1=3
INSTR_MEM[3] = 32'h4011_01B3; //sub x3=x2-x1=2-1=1
INSTR_MEM[4] = 32'h4020_8233; //sub x4=x1-x2=1-2=-1=complement ffff
_ffff
INSTR_MEM[5] = 32'h0011_61B3; //OR x3=x2||x1=2||1=3
INSTR_MEM[6] = 32'h0011_71B3; //AND x3=x2&x1=0
INSTR_MEM[7] = 32'h0011_41B3; //XOR x3=x2^x1=3
INSTR_MEM[8] = 32'h0010_9193; //SLLI x3=x1<<1=2
INSTR_MEM[9] = 32'h0010_D193; //SRLI x3=x1>>1=0
INSTR_MEM[10] = 32'h4010_D193; //SRAI x3=x1>>1=0
INSTR_MEM[11] = 32'h4012_5193; //SRAI x3=x4>>1=ffff_ffff>>1=ffff_ffff
INSTR_MEM[12] = 32'h4040_2283; //lw x5=mem[0+1]=mem[1]=820
INSTR_MEM[13] = 32'h0020_A323; //sw mem[x1+6]=mem[7]=x2=2
INSTR_MEM[14] = 32'h0070_2303; //lw x6=mem[x0+7]=mem[7]=2
INSTR_MEM[15] = 32'h0020_8463; //beq x1!=x2
INSTR_MEM[16] = 32'h0032_0463; //beq x3=x4,pc+=8(1000)
INSTR_MEM[17] = 32'h0020_8463; //beq x1!=x2
INSTR_MEM[18] = 32'h0032_0463; //beq x3=x4,pc+=8(1000)
INSTR_MEM[19] = 32'h0080_036F; //jal x[6]=pc+4,pc+=8(1000) NOP
INSTR_MEM[20] = 32'h0080_036F; //jal x[6]=pc+4,pc+=8(1000)
INSTR_MEM[19] = 32'h0080_036F; //jal x[6]=pc+4,pc+=8(1000) NOP
INSTR_MEM[20] = 32'h0080_036F; //jal x[6]=pc+4,pc+=8(1000)
INSTR_MEM[21] = 32'h0083_23E7; //jalr pc=x[6]+8,x[7]=pc+4 NOP
INSTR_MEM[22] = 32'h0083_23E7; //jalr pc=x[6]+8,x[7]=pc+4
INSTR_MEM[23] = 32'h0000_1417; //auipc x[8]=pc+1_0000_0000_0000(1)
```

Next, we will select some instructions to explain the operation of the previous three types of instructions:

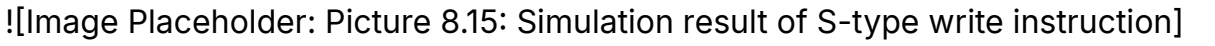
R-type Calculation Instruction: `INSTR_MEM[0] = 32'h0010_0093; //addi x1=x0+1=0+1=1`
When this instruction is read, it is first sent to the Control Unit for parsing, sending control signals to other parts. At the same time, Register File and Immediate Data will read the required `rs1` and immediate value and complete the corresponding operations. Then the ALU will receive `rs1` and the immediate value as `Src1` and `Src2` respectively. Finally, the calculated result `ALUResult` will be written back to the register `rd` location.
For the PC, it will output `PC+4` to run the next instruction. The Shifter will not operate.
The relevant simulation results are below:

R-type Shift Instruction: `INSTR_MEM[8] = 32'h0010_9193; //SLLI x3=x1<<1=2`
When this instruction is read, it is first sent to the Control Unit for parsing, sending control signals to other parts. At the same time, the Register File will read the required `rs1` and complete the corresponding operations. Then the Shifter will receive this data and the instruction, extract the corresponding shift data size, and complete the shift requirement. Then, the shifted result `ShOut` is input as `Src1` of the ALU, and for Immediate Data, we specifically default the shift number of the shift instruction to 0, so that the addition performed in the ALU will not change the result of only shifting. Finally, the result `ALUResult` calculated by the ALU will be written back to the register `rd` location.
For the PC, it will output `PC+4` to run the next instruction.
The relevant simulation results are below:

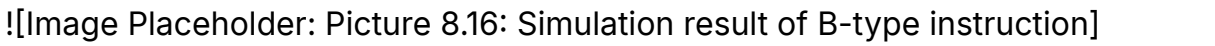
L-type Load Instruction: `INSTR_MEM[12] = 32'h4040_2283; //lw x5=mem[0+1]=mem[1]=820`
When this instruction is read, it is first sent to the Control Unit for parsing, sending control signals to other parts. At the same time, Register File and Immediate Data will read the required `rs1` and immediate value and complete the corresponding operations. Then the ALU will receive `rs1` and the immediate value as `Src1` and `Src2` respectively. Finally, the calculated result `ALUResult` will be sent to the Data Memory for reading the corresponding address. The data read out will be written back to `rd` as the final `Result`.
For the PC, it will output `PC+4` to run the next instruction. The Shifter will not operate.
The relevant simulation results are below:

S-type Store Instruction: `INSTR_MEM[13] = 32'h0020_A323; //sw mem[x1+6]=mem[7]=x2=2`
When this instruction is read, it is first sent to the Control Unit for parsing, sending control signals to other parts. At the same time, Register File and Immediate Data will read the required `rs1` and immediate value and complete the corresponding operations. Then the ALU will receive `rs1` and the immediate value as `Src1` and `Src2` respectively. Finally, the calculated result `ALUResult` will be sent to the Data Memory together with `rs2` for reading and replacing the corresponding address.
For the PC, it will output `PC+4` to run the next instruction. The Shifter will not

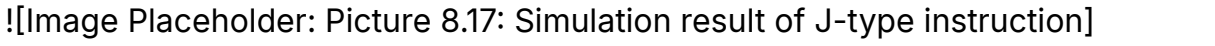
operate.
The relevant simulation results are below:



B-type Instruction: `INSTR_MEM[16] = 32'h0032_0463; //beq x3=x4,pc+=8(1000)`
When this instruction is read, it is first sent to the Control Unit for parsing, sending control signals to other parts. At the same time, the Register File will read the required `rs1` and `rs2` and complete the corresponding operations. Then the ALU will receive `rs1` and `rs2` as `Src1` and `Src2` respectively. We will perform subtraction to compare these two values to determine if they are 0, i.e., if they are equal. If these two values are equal, the `zero` output by the ALU is valid.
For the PC, since it receives the valid control signal from `BEQ` and `zero`, it simultaneously receives the immediate value information in the Immediate Data and performs addition processing with the PC. Finally, it will output in the form of `PC+8` written in the code to skip the next instruction. The Shifter will not operate.
The relevant simulation results are below:

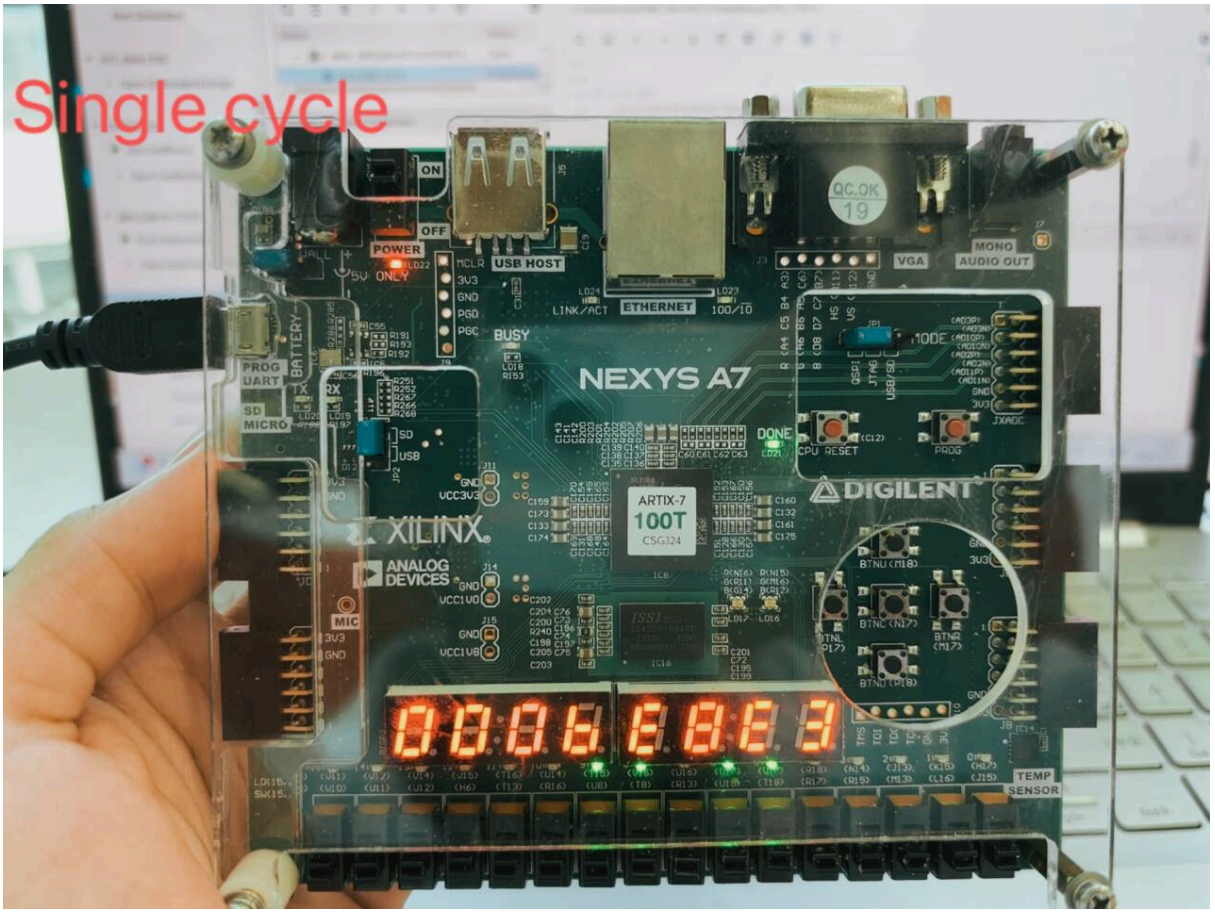


J-type Instruction: `INSTR_MEM[20] = 32'h0080_036F; //jal x[6]=pc+4,pc+=8(1000)`
When this instruction is read, it is first sent to the Control Unit for parsing, sending control signals to other parts. At the same time, Register File and Immediate Data will read the required `rd` and immediate value and complete the corresponding operations. Afterward, we will skip the ALU and reference `PC+4` directly from the PC MUX as our `Result` to write back to the register `rd`.
For the PC, we will input the prepared immediate value and perform addition processing. Finally, it will skip the next instruction in the way of outputting `PC+8` written in the code. The Shifter will not operate.
The relevant simulation results are below:

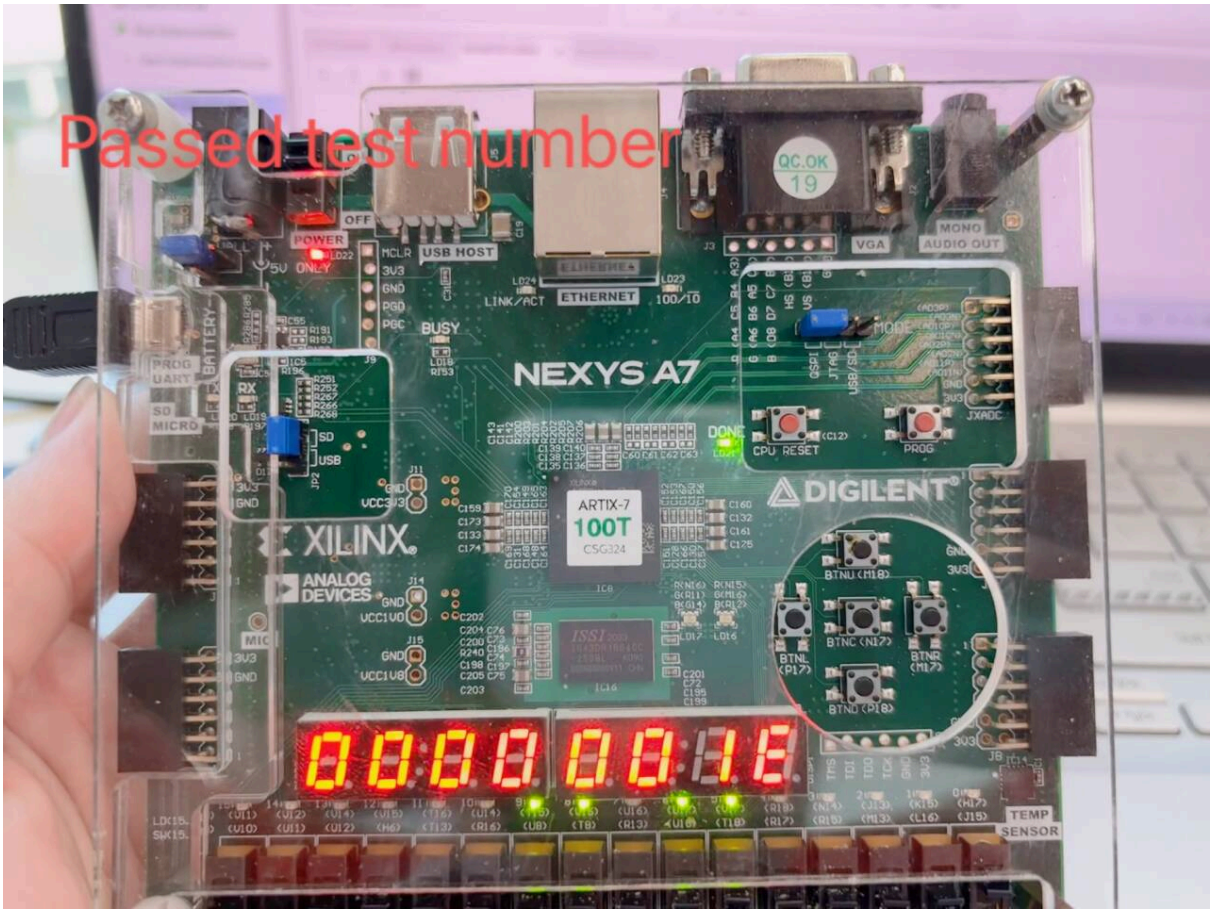


1.5 Board test results for Single-cycle CPU

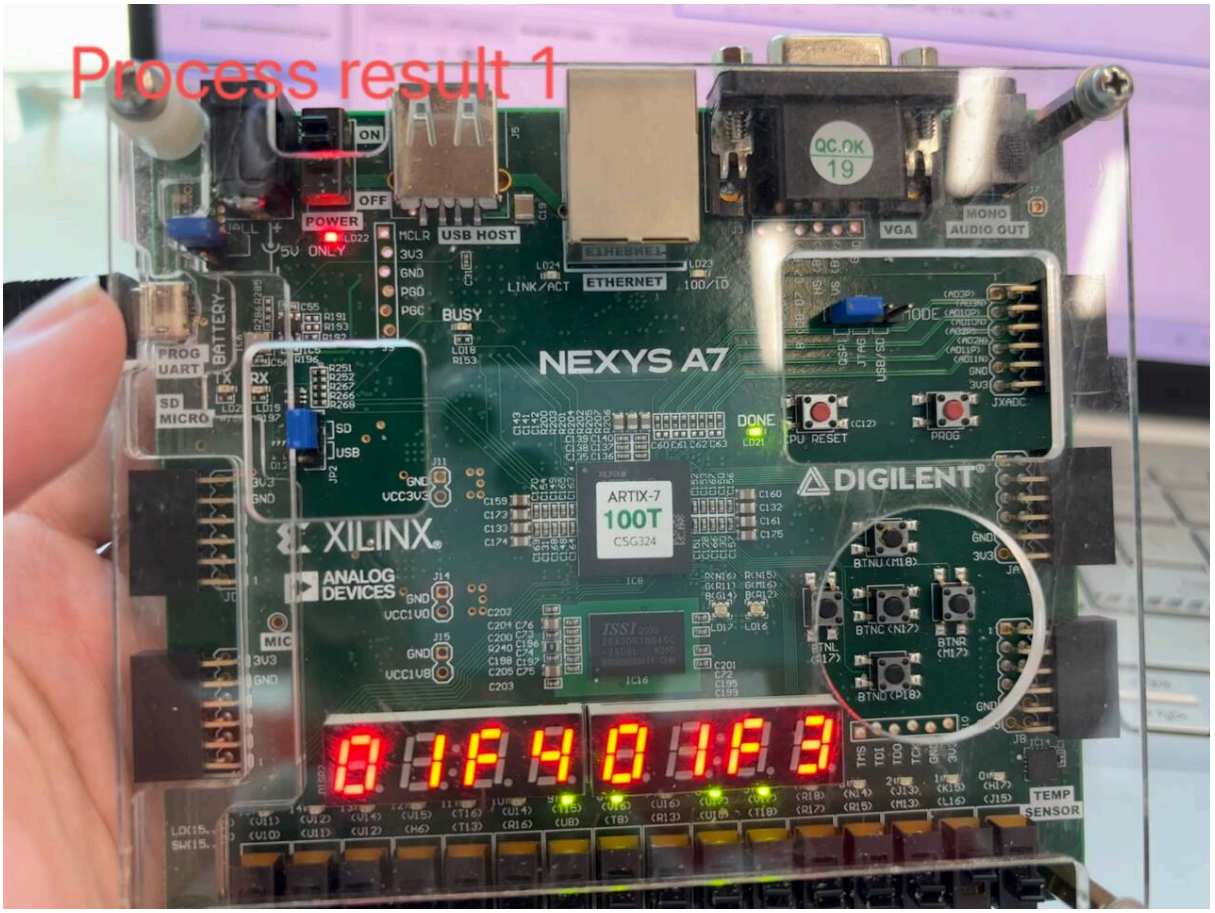
cpu time



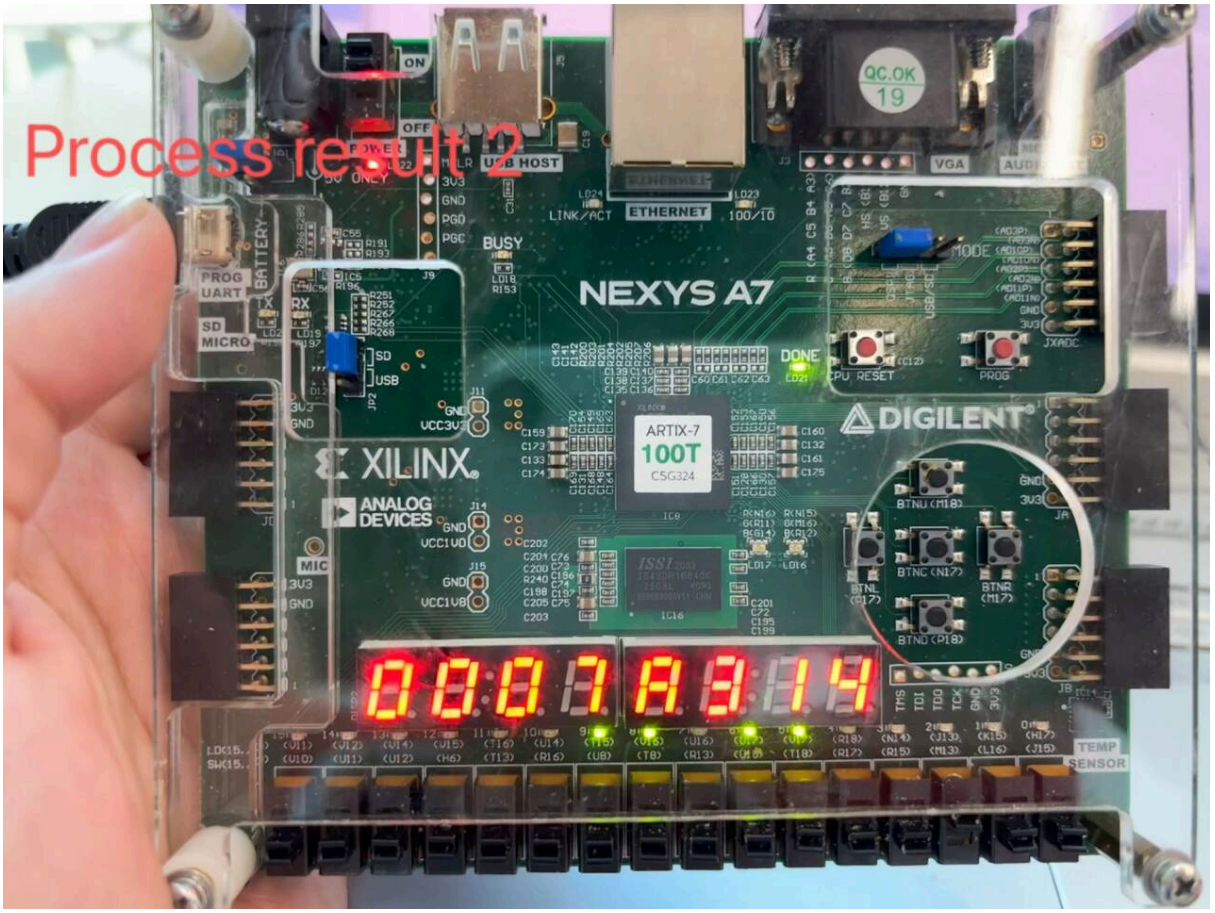
passed test number



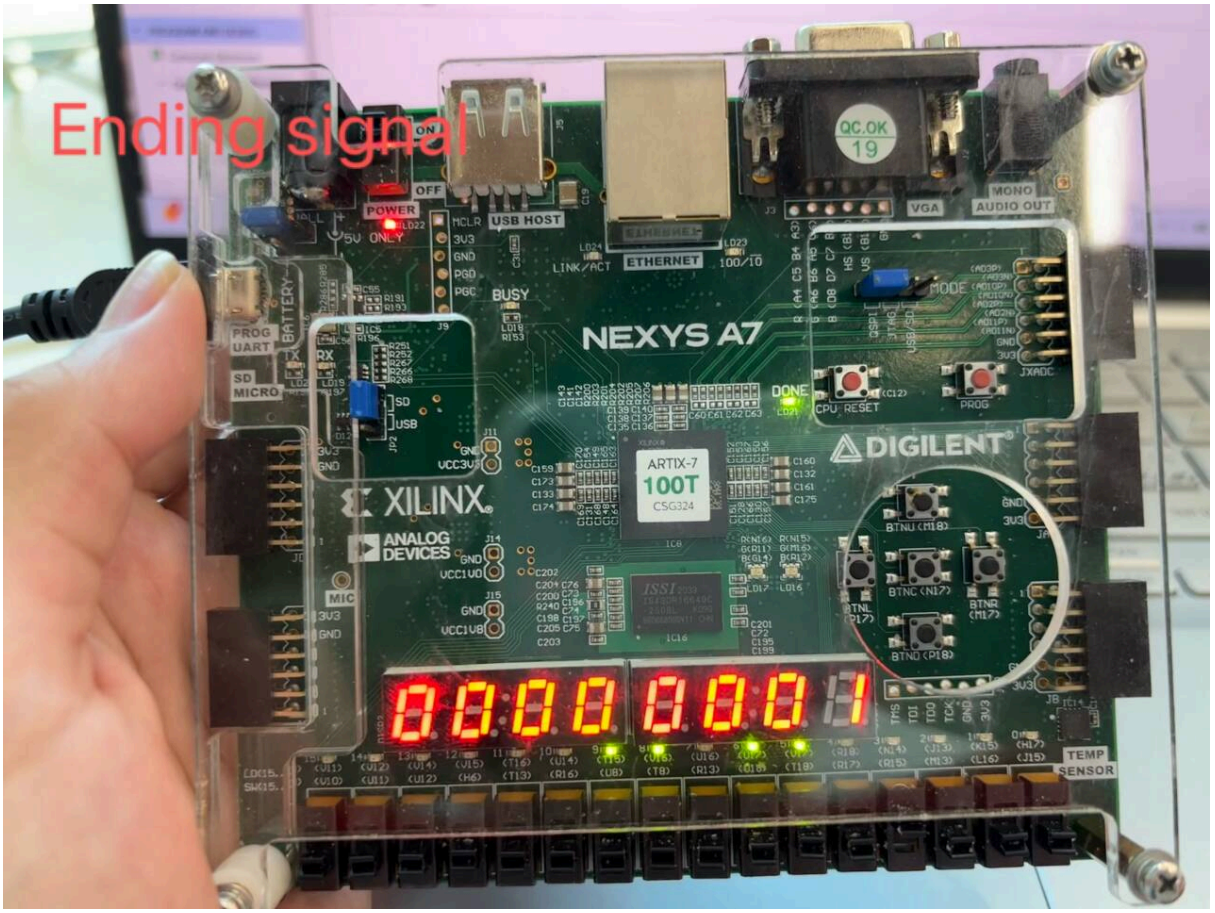
process result1



process result2



ending signal



1.6 Difficulties and Harvests of the Implementation Process

1.6.1 Difficulties Encountered

Writing Truth Tables and Code Parsing

The most complex part of this project is the writing of the truth table in the Control Unit. First, we need to classify according to the final implementation results of each instruction, reducing the required control commands as much as possible, and condensing and fusing common functions. Through the functional analysis of the instruction set, we learned the special functions of some codes: `x=PC+4`, `pc+=offset`, `pc=x+offset`, `x=PC+imm(<<12)`. Therefore, we specially set control signals in the Control Unit with their functional definitions. In addition, we found that we cannot directly set B-type instructions in the

same form as Jal, so we set two signals for them separately. When such codes are sent for parsing, corresponding control signals are activated and transmitted to the PC MUX, and then detected in combination with the corresponding judgment condition of `zero`.

![Image Placeholder: Picture 8.20: RISC-V Instruction Set]

PC MUX
In addition, the PC MUX is also relatively complicated. For this reason, we separate the PC update and MUX into two parts for better logical analysis. Previously in the writing of the truth table, we mentioned `x=PC+4`, `pc+=offset`, `pc=x+offset`, which are variables related to the PC. Therefore, in addition to considering the functional implementation of the PC's own update in the MUX, we also need to consider introducing or extracting some special values to meet the needs of various instructions in different modules. Therefore, in this part, we first need to prepare various inputs: current `PC`, `imm` and other variables. At the same time, we can borrow the ALU to implement some of the calculations, reducing the burden on the hardware through logic.

Result MUX
In addition to the PC MUX, we also involved the design of the MUX in the Result MUX. Similarly, previously in the writing of the truth table, we mentioned `x=PC+4`, `pc=x+offset`, `x=PC+imm(<<12)`, which are variables related to `x`. Therefore, when designing, we need to think about where is appropriate to introduce or extract. Similarly, whether we can borrow other modules to implement partial operations to reduce burdens in other aspects.

1.6.2 Final Takeaways

Understanding of Architecture
Compared to the ARM architecture known from textbooks, this RISC-V architecture requires us to design it ourselves based on relevant experience. The whole process is not simple, but we can still start from the skeleton of the architecture, those fixed modules, and start from the implementation of simple functions; then add flesh and blood—Extended Function. The whole process was certainly not smooth sailing; too many new variables were quite a shock (see code parsing in Difficulties). Where they belong, where input and output go—when these were laid out one by one, it was like the clouds clearing to see the moon, drawing the final architecture image.

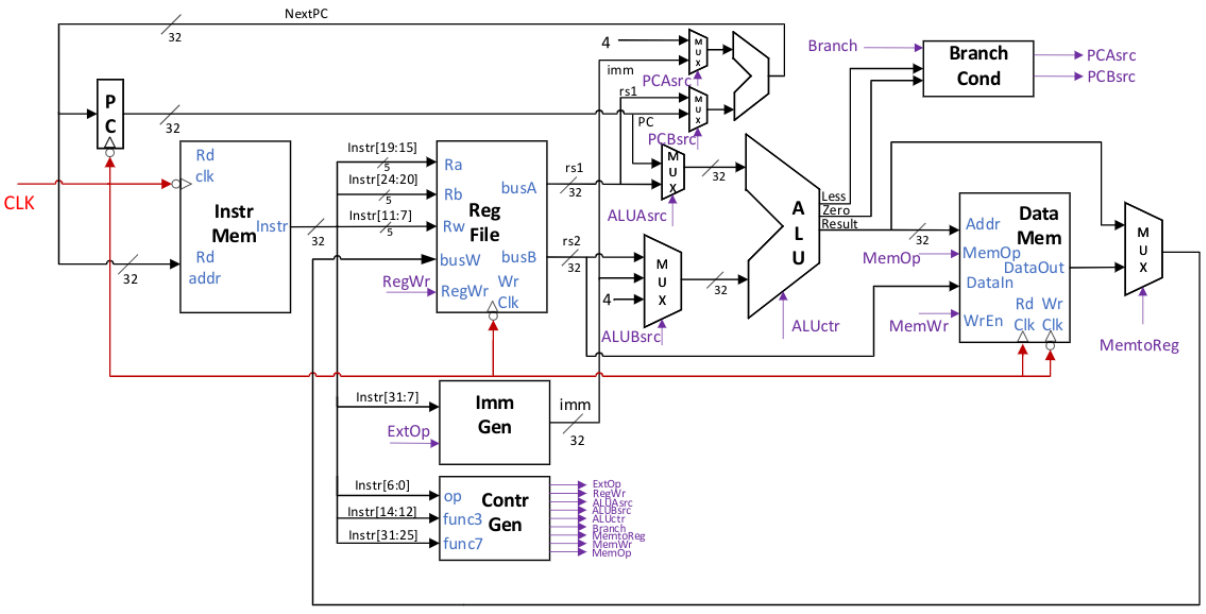
Understanding of Code
Through this project, we learned how to write the truth table after code parsing

by ourselves. This process is undoubtedly difficult. But peeling the cocoon in logic is also an unforgettable memory. through careful classification and multiple modifications, and finally after counting several adjustments, we realized the current truth table. While achieving the specified goal, we also reduced a lot of code, which is very proud.

Mutual Help Between Groups

In the whole process, some initial code writing was too complicated, not only logically complex, but also difficult to debug due to too many lines of code. Fortunately, we were not doing it alone. In logic, we communicated with each other and carried out effective simplification; in code, we learned from each other and carried out effective reduction. Therefore, in the end, our code can be simple enough and the layout can be exquisite enough, and our program can run smoothly and perfectly.

Chapter 2: Five-Stage Pipelined RISC-V CPU Implementation



2.1 Introduction to Five-Stage Pipelined RISC-V CPU

To further improve the performance of the RISC-V processor, we adopted a five-stage pipeline design similar to the ARM architecture previously. By adding registers written on the rising edge of the clock to the single-cycle architecture,

a time process is artificially divided into five time processes, including Instruction Fetch (F stage), Instruction Decode (D stage), Execution (E stage), Memory Read and Write (M stage), and Write Back (W stage). We will explain the division of labor for each stage in detail in the following introduction.

The five-stage pipeline design can improve the throughput of the processor executing instructions and reduce the clock cycle, thereby achieving the effect of improving IPC (Instructions per Cycle) and processor performance. In an ideal situation, the performance improvement of a five-stage pipelined processor compared to a traditional single-cycle processor should be 5 times, because the processor can execute different stages of five instructions in the same cycle. However, considering that the time of one cycle of the pipeline is not one-fifth of the original single-cycle time (as shown in Figure 2.1), because this is limited by the slowest executing stage among the five stages (usually the data read/write stage, the time required for this stage is greater than one-fifth of the single-cycle time), the clock cycle of the pipelined processor is often greater than one-fifth of the original single cycle, but it still has a significant performance improvement, about three times.

However, multi-cycle processors come with additional issues that need attention, including Structure Hazard, Data Hazard, and Control Hazard between adjacent instructions. To solve these problems, we introduced a Data Hazard module in the multi-cycle RISC-V processor to detect conflicts caused by instruction parallelism and provide corresponding solutions.

The design of the five-stage pipelined RISC-V processor in this chapter is mainly designed for the RV32I base instruction set, and all data is 32 bits. The instruction set content achievable by this CPU is shown in Figure 2.2.

In the following chapters, we will describe in detail the detailed architecture of the RISC-V Pipeline, the solutions adopted for Hazards, specific execution plans, and the instruction simulation results of the RISC-V Pipeline.

2.2 Detailed Explanation of Five-Stage Pipelined RISC-V CPU Architecture

This section combines Figure 2.3 to briefly introduce the component modules of the five-stage pipelined RISC-V processor and the basic functions of each module. However, the Hazard processing part will not be explained in detail in this section but will be discussed in Section 2.3.

2.2.1 Instruction Fetch

F stage is the first stage experienced by instruction execution. Its main function is to control the address of the current instruction and fetch the instruction content under that address in Instruction Memory. At the same time, the address jump is controlled by the PCsrc signal. Under normal circumstances, the instruction address jumps directly to the next one, unless a Branch instruction takes effect (takes effect in E stage). The calculation of the PC address in the F stage is integrated into the PC module (PC.v) [1] in the code, and the calculation result of the PC module is given to Instruction Memory to obtain the corresponding 32-bit instruction.
[1] Parentheses indicate the corresponding module in the code.

2.2.2 Clock-Controlled Pipeline Registers

Pipeline registers are controlled by the clock and write content only on the rising edge of the clock, thereby separating the F stage and D stage executing the same instruction in time, and executing different instructions in F stage and D stage at the same time. Other registers between adjacent stages also play the same role. The relevant information of the instruction executed at each level is stored in the register between it and the previous level, and the result of the execution at each level is passed along with the relevant information of the instruction (such as control signals) to the next stage (next level) to continue execution in the next cycle. Finally, it is achieved that each level executes different instructions in the same cycle, that is, five instructions can be executed simultaneously, with the instruction in the W stage being the earliest started instruction and the instruction in the F stage being the latest started instruction.

2.2.3 Instruction Decode

The D stage mainly decodes the instruction read by the F stage in the previous cycle, prepares the data objects needed for the operation, and the control

signals controlling the execution method of subsequent stages. It mainly includes the following three modules:

- **Registers File (register_file.v):** This module mainly reads the contents of the registers pointed to by rs1 and rs2 in the instruction. Later in the W stage, content is written to the register pointed to by rd.
- **Control Unit (control.v):** This module generates control signals through instructions. The control signals will follow the data path backwards and be used in the corresponding stages. For different RV32I base instructions, see Table 2.1 for the generation of control signals.
- **Imm Gen (imm_gen.v):** This module extends the immediate value in the instruction to a 32-bit immediate value generation module. Among all instructions containing immediate values, only `sltiu` requires unsigned extension.

2.2.4 Execute

The E stage performs corresponding operations on the data generated by the D stage in the previous cycle under the action of control signals. Also, in the E stage, it can be judged whether the jump instruction needs to jump. If no jump is needed, continue to execute the subsequent instructions; if a jump is needed, this will lead to a Control Hazard because instructions that do not need to be executed have already been executed. Specific solutions can be found in 2.3. The E stage mainly includes the following modules:

- **ALU module (ALU.v):** This module performs corresponding operations on two 32-bit input data (src1 and src2). The operation performed by the ALU is controlled by ALUop. There are ten operation types: `4'b0000` corresponds to ADD; `4'b0001` corresponds to SUB; `4'b0010` corresponds to XOR; `4'b0011` corresponds to OR; `4'b0100` corresponds to AND; `4'b1000` corresponds to logical left shift (`src1<<src2`); `4'b1001` corresponds to logical right shift (`src1>>src2`); `4'b1010` corresponds to arithmetic right shift; `4'b1011` corresponds to comparison (`src1<src2 ?`); `4'b1100` corresponds to unsigned comparison.
- **Branch PC Calculate (branch_PC.v):** This module mainly takes effect in jump instructions, calculating the address to jump to, and passing it to the F stage after the cycle ends to complete the jump. However, due to the particularity of the `jalr` instruction, the jump instruction target is `rs1+imm`, so when calculating the jump PC, the operation object is controlled by the

`branch src` signal. Under normal circumstances, `branch src` is 0, and only under the `jalr` instruction is it 1 (see Architecture Diagram 2.3 for details).

- **Branch Control (branch_control.v):** The function of this module is to determine whether the current instruction needs to jump. If a jump is needed, it will output a signal `PCsrc=1`, and the jump will be completed when the rising edge of the next cycle arrives; if no jump is needed, `PCsrc=0`. In the input signals of this module, the `branch` control signal determines whether the signal is a jump instruction, and the `branch type` signal combined with the ALU calculation result determines whether the jump condition is met, and finally decides whether to jump. For `branch type`: `2'b00` corresponds to jump when equal (when `reg write` signal is 0); `2'b01` corresponds to jump when not equal; `2'b10` corresponds to jump when less than; `2'b11` corresponds to jump when greater than or equal to. But note that when `branch type` equals `2'b00` and `reg write` is 1, it is `jal` and `jalr` instructions, which are unconditional jumps.

It can be noted that the data entering the ALU is also selected, controlled by `ALUsrc1` and `ALUsrc2` signals. This is considering that in `jal`, `jalr`, `lui`, and `auipc` instructions, the data operation written back to rd does not come from `imm` or the content read from the register file, but from PC, `imm<<12`, etc. [2], so a selection is made.
At the same time, due to the Data Hazard introduced by the pipeline, the data read from the register file may be "outdated" data, which needs to be solved by the Data Forwarding method introduced later.
[2] Different instructions corresponding to different src1 and src2 can be analyzed by combining the architecture diagram and the control signals generated by the instructions.

2.2.5 Memory Read/Write

The M stage mainly performs read/write operations for instructions that require memory read/write. Since RV32I contains reads/writes of different Byte counts, `mem read` and `mem write` control signals carry out relevant control.

- `mem write` signal: `2'b00` corresponds to no writing to memory; `2'b01` corresponds to byte-level memory write; `2'b10` corresponds to half-word-level memory write; `2'b11` corresponds to word-level memory write.
- `mem read` signal (3 bits): First, for the lower two bits of control signals, `2'b00` represents no memory reading; `2'b01` represents byte-level reading; `2'b10` represents half-word-level reading; `2'b11` represents word-level reading. For

the highest bit control signal, it mainly controls whether signed extension is performed when the read data is extended to 32 bits during non-whole word reading. 0 corresponds to signed extension, 1 corresponds to unsigned extension.

However, also due to the Data Hazard introduced by the five-stage pipeline, the written content may be "outdated" [3], which can also be solved by Data Forwarding, which will not be repeated here.
[3] Such as Load and Store

2.2.6 Write Back

The W stage is the last stage of the five-stage pipeline. Its main function is to write the required content back to the registers file (ALU calculation result / Data Memory read data, controlled by the `Mem2Reg` signal). Its function is relatively single and simple to understand.

2.2.7 Summary

So far, there is a relatively clear concept of the basic architecture of the five-stage pipelined RISC-V processor. In short, the five-stage pipeline divides an original instruction into five stages for execution. Hardware of different levels executes different stages of different instructions, which can maximize the use of existing processor hardware. But at the same time as building the five-stage pipeline, we found problems brought by Data Hazard and Control Hazard. In the following section 2.3, we will discuss in detail how to solve them.

![Image Placeholder: Table 9.1: Control Signals Generated by RV32I Instructions]

2.3 Hazard Analysis and Resolution

In the five-stage pipelined RISC-V, the main problems encountered can be divided into three categories, including Structure Hazard, Data Hazard, and Control Hazard. In the overall architecture, we introduced the Hazard Detector (`Data_Hazard_Detector.v`) [4] module to detect the occurrence of Hazards and solve problems in time by giving control signals. Next, we will explain in detail how to solve various Hazard problems.
[4] Named Data Hazard detector module when writing code, but actually also detects Control Hazard, do not be misled by the name.

2.3.1 Structure Hazard Analysis and Resolution

In a word, We avoid structural risks by storing instruction memory and data memory separately through different hardware.

In the original single-cycle processor, data (including data and instruction) reading and writing do not happen at the same time, so there will be no simultaneous reading and writing of memory resources. In the five-stage pipeline, since each level works at the same time, reading and writing from Memory must happen at the same time. For example, while the F stage reads instructions, the M stage may be writing or reading data, resulting in Memory resources being occupied by two different instructions at the same time. This is Structure Hazard.

The way to solve Structure Hazard is "Harvard Architecture". "Harvard Architecture" is a variant of "Von Neumann Architecture", separating instruction memory and data memory so that instruction reading and data reading/writing can be carried out simultaneously. "Harvard Architecture" is also the architecture we have always adopted in the experimental course, so Structure Hazard does not need additional consideration in this project.

2.3.2 Data Hazard Analysis and Resolution

Since the data required to execute an instruction cannot be fully provided by the 32-bit instruction itself, it needs to be read from the registers in the Registers File, and the data in the registers may change at any time. However, due to the classification mechanism of the five-stage pipeline, when the previous instruction has not been completed, the subsequent instruction has already started execution. When the subsequent instruction needs to use the execution result of the previous instruction, since the previous instruction has not completed execution, the subsequent instruction cannot obtain the correct data in the register. This is Data Hazard, also called Data Dependency.

To solve the above problem, we can call the results of previous instructions in advance, because these results are often not obtained in the final stage, but the final result to be written into the Registers File has already been obtained after the E stage or M stage ends. We can transfer the data of the later stage being executed in the five-stage pipeline to the earlier stage through Data Forwarding, which can also be understood as forwarding the result of the previous instruction to the subsequent instruction in advance. This is the basic idea of solving Data Hazard.

In Data Hazard, instructions that may have data dependencies are generally adjacent instructions, instructions separated by one instruction, and

instructions separated by two instructions. This is because the pipeline processes only five instructions at the same time. The maximum interval between two simultaneously executing instructions is three instructions, that is, the instruction in the F stage has data dependency on the instruction in the W stage, but this will not happen because the W instruction changes the data in the Registers File in the D stage, which has no effect on the instruction address. Next, we will discuss the problems caused by data dependencies between different instructions and their corresponding specific solutions.

2.3.2.1 Data Hazard 1: Adjacent Instructions & Load and Store

For Load and Store type instructions, the data read by the Load instruction will be immediately written into Memory as storage data by the next Store instruction. The Load data cannot be stored in the Register File in time. The content of rs2 read by the Store instruction in the D stage can be considered wrong, or understood as "outdated", because what is actually needed is the Read Data of the Load instruction.

Solving this Data Hazard can forwarding the input of the W stage of the Load instruction to the M stage of the Store instruction, as shown by the red arrow in Figure 2.4. Because these two stages of the two instructions start execution in the same cycle, the input of the W stage of Load is the content read in its M stage, which is also the content to be written back to the register later, and is the correct content that the Store instruction needs to store in Memory. Here it is utilized in advance without being written back.

![Image Placeholder: Picture 9.4: Adjacent Instructions & W Forwarding to M]

In actual code operation, this Hazard is detected through the Hazard Detector. When the instruction executed in the M stage is a memory write instruction (`mem_write_M != 2'b00`) and its rs2 is the same as the rd of the instruction executed in the W stage (`rs2_M==rd_W`), and the instruction in the M stage writes the data read from memory back to the register (`Mem2Reg_W==1 && reg_write_W==1`), this Hazard appears, and the corresponding control signal `WriteData_src` is set high to select the W stage forwarding data in the Mux before the Memory data write port in the M level of architecture diagram 2.3. The detailed code is as follows:

```
module hazard_detection_unit(
    input [2: 0] ex_readMem,
    input [4: 0] ex_rd, id_rs1, id_rs2,
```

```
    output reg pause
);

always @(*) begin
    pause = 1'b0;
    if((ex_readMem != 3'b000) && ((ex_rd == id_rs1) || (ex_rd == id_rs2))) be
gin
        pause = 1'b1;
    end
end

endmodule
```

2.3.2.2 Data Hazard 2: Adjacent Instructions & Execute and Use

Another possible Data Hazard between adjacent instructions is that the data operated by the second instruction in the E stage (i.e., the data read from the register in the D stage in the previous cycle) is "outdated" data. The first instruction has already recalculated this data. This is called Execute and Use.

Because the first instruction has already obtained the result to be written back through ALU calculation in the E stage, then when the first instruction enters the M stage (i.e., the second instruction enters the E stage), Data Forwarding is performed at the beginning of the cycle, forwarding the data of the M stage to the E stage. As shown by the red line in Figure 2.5, at this time, the data operated by the second instruction in the E stage is re-selected by the control signal to obtain the correct data.

![Image Placeholder: Picture 9.5: Adjacent Instructions & M Forwarding to E]

In the actual code, the Hazard detection logic is: when finding that the rd of the M stage of the first instruction (which eventually needs to write back to the register `reg_write_M`) is the same as the rs1 or rs2 required by the second instruction (`rd_M==rs1_E/rs2_E`), it indicates that the data read by the second instruction from the Registers File is "outdated". Generate selection signal `rs1_src` or `rs2_src`, and select the data forwarded from M stage to E stage in the Mux before ALU.

Of course, you may have a question here. The judgment condition does not judge whether the data written back by the first instruction comes from the ALU result or the Memory result. Why are you sure that the data passed from the M

stage is correct? For example, when the first instruction is a Load instruction, the correct data exists only after the M stage ends. This is actually Data Hazard 2.3.2.3 analyzed in the next section, and this Hazard has already been detected in the E stage of the first instruction (i.e., the D stage of the second instruction), which is the previous cycle of the cycle we are discussing now, so we don't need to worry about the case where the first instruction is a Load instruction in this cycle. The detailed Verilog code is as follows:

```
// Detection Part (Data_Hazard_Detector.v)
// Only detects M forwarding to E part
always @(*) begin //rs1_src; M stage forward to E stage. Only showing rs1_
src part, rs2_src is similar
    if (rd_M==rs1_E && reg_write_M) begin
        rs1_src=2'b01; //M forwarding to E
    end
    else begin
        rs1_src=2'b00;
    end
end
//-------------------------------------------------------------------------
------------------------
// E stage data selection part (RISC-V.v)
always @(*) begin //hazard src1, only showing src1 part, src2 is similar
    if(rs1_src==2'b00)begin
        read_data1_E=read_data1_D2E;
    end
    else if (rs1_src==2'b01) begin
        read_data1_E=ALUResult; //ALUResult is the input signal of M stage, fr
om calculation result of E stage in previous cycle
    end
    else begin
        read_data1_E=read_data1_D2E;
    end
end
```

2.3.2.3 Data Hazard 3: Adjacent Instructions & Load and Use

Combining the problem we raised in the previous section 2.3.2.2, we need to solve the third type of Data Hazard. This Hazard is because the operation data needed by the second instruction comes from the first data, and the first one is a Load instruction. We call this Hazard Load and Use.

If we still want to adopt the same Data Forwarding solution as in the previous two sections, we need to forwarding the input data of the W stage of the first instruction to the E stage of the second instruction. This is impossible, as shown by the red line in Figure 2.6, because these two stages are not performed in the same cycle (at the same moment), and the data transmission path is reverse-time.

To solve this problem, we can only artificially delay the E stage of the second instruction by one cycle, aligning the W stage of the first instruction with the E stage of the second instruction to complete W Forwarding to E, as shown in Figure 2.7. We call this process instruction Stall.

During Stall, the second instruction seems to be stagnant in the D stage for one cycle. At the same time, it should be noted that not only the second instruction is stagnant, but the F stage of the third instruction is also stagnant for one cycle, and the fourth instruction enters the processor one cycle later than originally expected. This is because the order of instruction execution cannot be changed, otherwise problems like data overwriting will occur. The zero-th instruction (currently in the M stage) is not obstructed and will enter the W stage normally in the stall cycle.

During Stall, our idea is that the content of the F and D levels in the Stall cycle is the same as the previous cycle (requires Stall signal), and the M and W levels take over the tasks of the relevant instructions in the previous cycle and continue to execute (proceed as usual). It is not difficult to find that at this time, the E level has no content to execute. We must ensure that the E level does not produce any data after the Stall cycle ends, and does not pass any data to the M level in the next cycle. Then in the Stall cycle, the E level needs a clear signal, that is, a Flush signal.

In code implementation, when Data Hazard is detected when the first instruction executes to the E stage and the second instruction executes to the D

stage, detecting that the data needed by the second instruction later in the E stage is the content read and written back by the first instruction in the M stage (`rd_E==rs1_D(rs2_D) && reg_write_E && Mem2Reg_E`), Hazard Detector sets `stall_F` , `stall_D` and `Flush_E` signals high, taking effect when the clock edge rises in the next cycle. At the same time, in the cycle after Stall, data forwarding between W stage and E stage is also detected, generating control signals `rs1_src` and `rs2_src` for the Mux before ALU (the same control signal acting as in the previous section, because it controls the same Mux). Implementing Stall of PC address requires PC not to change during PC update. Similarly, Stall of D stage means registers between F level and D level do not update when the clock arrives. When performing Flush on E level, it means setting the content in the registers between D level and E level to zero when the clock of the Stall cycle arrives. The detailed code is as follows:

```
// Detection Part (Data_Hazard_Detector.v)
// Only detects W forwarding to E part
always @(*) begin //rs1_src; W stage forward to E stage. Only showing rs1_
src part, rs2_src is similar
    if(rd_W==rs1_E && reg_write_W)begin
        rs1_src=2'b10; //W forwarding to E
    end
    else begin
        rs1_src=2'b00;
    end
end

wire match_load_and_use=(rs1_D==rd_E)||(rs2_D==rd_E);
assign stall_F=match_load_and_use && Mem2Reg_E && reg_write_E && (~R
eset); //stall_F; load and use (D stage and E stage) (Need stall and flush, st
all F stage and D stage, flush E stage content)
assign stall_D=match_load_and_use && Mem2Reg_E && reg_write_E && (~R
eset); //stall_D; load and use (D stage and E stage) (Need stall and flush, st
all F stage and D stage, flush E stage content)

assign flush_E=(match_load_and_use && Mem2Reg_E && reg_write_E) &&
(~Reset)
//-----------------------------------------------------------------------
-----------------------
```

```
// F stage address jump (PC.v)
always @(posedge clk or posedge reset) begin
    if (reset==1) begin
        current_PC<=32'b0;
    end
    else if (stall_F==1) begin //Effect of stall signal, PC does not add 4, i.e., n
ext instruction is still this instruction
        current_PC<=current_PC;
    end
    else begin
        current_PC<=next_PC;
    end
end
//-------------------------------------------------------------------------
-----------------------
// D stage stall implementation (RISC-V.v)
always @(posedge CLK or posedge Reset) begin
    if(Reset==1 || flush_D)begin //flush signal, irrelevant to this part, solves C
ontrol Hazard problem
        instr_F2D<=32'b0;
        PC_F2D<=32'b0;
    end
    else if(stall_D)begin //stall, register update between F level and D level wi
ll not change
        instr_F2D<=instr_F2D;
        PC_F2D<=PC_F2D;
    end
    else begin
        instr_F2D<=Instr;
        PC_F2D<=PC;
    end
end
//-------------------------------------------------------------------------
-----------------------
// E stage Flush implementation (RISC-V.v)
always @(posedge CLK or posedge Reset) begin
    if(Reset == 1 || flush_E)begin //When flushing is like reset, all control sign
als, all transmission data become 0.
```

```
        ALUsrc1_D2E<=2'b0;
        ALUsrc2_D2E<=2'b0;
        ALUop_EX_D2E<=4'b0;
        // ... (setting other signals to 0) ...
        read_data1_D2E<=32'b0;
        read_data2_D2E<=32'b0;
        PC_D2E<=32'b0;
    end
    else begin
        ALUsrc1_D2E<=ALUsrc1_D;
        ALUsrc2_D2E<=ALUsrc2_D;
        ALUop_EX_D2E<=ALUop_EX_D;
        // ... (setting other signals to values from D stage) ...
        read_data1_D2E<=read_data1_D;
        read_data2_D2E<=read_data2_D;
        PC_D2E<=PC_D;
    end
end
```

2.3.2.4 Data Hazard 4: Single Instruction Gap & Write and Use

Besides Data Hazard occurring between adjacent instructions, data dependencies can also occur between two instructions separated by one instruction. I call the relationship between these two instructions "single instruction gap". When the data operated by the ALU unit of the third instruction is the data finally written back by the first instruction, the register content read by the third instruction in the Registers File is "outdated" data. This relationship between the two instructions is called Write and Use.

The way to solve this Hazard can use a method similar to the previous one. The E stage of the third instruction and the W stage of the first instruction are executed at the same time. At this time, the first instruction must have obtained the correct write-back data. We can forwarding it to the E stage of the third instruction in advance, allowing the third instruction to obtain the correct data, as shown in the figure. And in this case, we don't need to discuss whether the second instruction needs the E stage result of the first instruction (2.3.2.2 Adjacent Instructions & M Forwarding to E), or needs the M stage memory read content of the first instruction (2.3.2.3 Adjacent Instructions & W Forwarding to E) as before. Because when the E stage of the third instruction executes, the

first instruction has already entered the W stage. Regardless of whether the E stage or M stage obtains the correct result, the forwarding data at this time (the data about to be written back to the Registers File) must be correct. So we can consider that Write and Use includes Load and Store, Execute and Use, and Load and Use, these three situations, but can be solved by one data forwarding method.

In actual code implementation, when the Hazard Detector detects that the first instruction writes back to the register (`reg_write_W==1`), and the register written back is the register required to be read by the third instruction (`rd_W==rs1_E(rs2_E)`), it will generate corresponding `rs1_src` or `rs2_src` control signals to select the operation object of the E stage currently being performed by the third instruction, realizing data W forwarding to E. Detailed code is as follows:

```
// Detection Part (Data_Hazard_Detector.v)
// Only detects W forwarding to E part
always @(*) begin //Only showing rs1_src part, rs2_src is similar
    // ... (previous logic) ...
    else if(rd_W==rs1_E && reg_write_W)begin
        rs1_src=2'b10;
    end
    else begin
        rs1_src=2'b00;
    end
end
//------------------------------------------------------------------------
-----------------------
// E stage data selection part (RISC-V.v)
always @(*) begin //hazard src1, only showing src1 part, src2 is similar
    if(rs1_src==2'b00)begin
        read_data1_E=read_data1_D2E;
    end
    else if (rs1_src==2'b01) begin // M stage write-back data forwarding to E
stage, as ALU operation target
        read_data1_E=ALUResult;
    end
    else if (rs1_src==2'b10) begin // W stage write-back data forwarding to E
stage, as ALU operation target
```

```
        read_data1_E=write_data_W;
    end
    else begin
        read_data1_E=read_data1_D2E;
    end
end
```

In the code, we will find that the generation logic of `rs1_src` and `rs2_src` for Data hazard is the same as in 2.3.2.3 data hazard 3. This precisely reveals that the solution for data hazard 3 is essentially the same as data hazard 4. It's just that data hazard 3 needs to turn two adjacent instructions into two instructions with a single instruction gap when solving. So the stall signal and flush signal in data hazard 3 can be seen as inserting an empty instruction between the first instruction and the second instruction, so that the content read in the M stage of the first instruction can be forwarded to the E stage of the second instruction. At this time, the second instruction can be regarded as the third instruction.

Actually, 2.3.2.2 data hazard 2 and 2.3.2.1 data hazard 1 can also be solved by methods similar to data hazard 3, but this method will increase CPI, while the solutions adopted by data hazard 2 and data hazard 1 will not insert empty instructions and will not affect CPU performance.

2.3.2.5 Data Hazard 5: Double Instruction Gap & Write and Use

Finally, the two instructions that may generate data dependencies are instructions separated by two instructions, here referred to as "double instruction gap".
The fourth instruction may also have data reading errors due to the third instruction. But when the fourth instruction reads the content of Registers File, the first instruction is already in the write-back stage (W stage), but it will not be written until the rising edge of the clock in the next cycle arrives. At this time, the fourth instruction is too late to read the correct data. The relationship between these two instructions can also be seen as Write and Use.

Solving this problem does not require Data Forwarding. Just advance the writing of Registers File registers. So we write the content back to the register on the falling edge of the clock, which is equivalent to writing half a cycle in advance. The fourth instruction can read the correct data from the register in the second half of the D stage and pass it to the next stage to continue

executing instructions in the next cycle. The specific code is as follows, only need to change the update method of the register in Registers File:

```
always @(negedge clk or posedge reset) begin // Write on falling clock edg
e
    if(reg_write==1 && reset==0)begin
        case (write_add)
            // ...
        endcase
    end
    // ...
end
```

2.3.2.6 Discussion 1

RV32I instructions are divided into six types. Instructions that need to use data from registers include R-type, I-type, S-type, and B-type; instructions that change register contents are R-type, I-type, U-type, and J-type. when an instruction using data follows an instruction that changes registers, Write and Use Data Hazard may occur.

We can consider Execute and Use, Load and Use, Load and Store all belonging to Write and Use. The difference lies in that the correct data for Execute and Use is generated after the E stage of the first instruction ends, the correct data for Load and Use is generated after the M stage of the first instruction ends, while Load and Store is a special case of Load and Use where the correct data is generated in the M stage, but the use of data by subsequent instructions can be delayed to its M stage (which is the W stage of the first instruction).

Observing these instruction types, we can specifically list all possible situations of Data Hazard. Here only adjacent instructions and double instruction gap are considered. Triple instruction gap is not discussed as it does not involve Hazard Detector detection:

- **Load and Store:** I-type (Load type) + S-type (Only rs2 causes Data Hazard)
- **Load and Use (excluding Load and Store):** I-type (Load type) + R-type/S-type (Only rs1 causes Data Hazard)/B-type
- **Execute and Use:** R-type/I-type (non-Load, non-jalr)/U-type/B-type (no jump occurred) + R-type/I-type/S-type/B-type

Why emphasize non-occurring jumps for jump instructions? Because when a jump occurs, the following two instructions will be cleared, effectively becoming empty instructions, so there is no data dependency. However, during the actual processor operation, data dependency is also being detected and forwarding data, but it will be cleared later.

![Image Placeholder: Picture 9.9: RV32I Instruction Types]

We noticed that between I-type (Load type) and B-type, there are two types of Data Hazard, including Load and Use and Execute and Use, corresponding to Data Hazard caused by rs2 and rs1 of S-type respectively. Among them, only Load and Use needs to generate Stall in the case of adjacent instructions, so regarding the generation of Stall signal groups, we need to pay special attention to S-type, which is only generated when `rs1_D==rd_E`. Therefore, we modified the conditions for generating Stall signals in section 2.3.2.3. The specific code is as follows:

```
reg match_load_and_use_r;
wire match_load_and_use=match_load_and_use_r;

always @(*) begin
   if(rs1_F2D==rd_D2E)begin
      match_load_and_use_r=1;
   end
   else if(rs2_F2D==rd_D2E)begin
      if (mem_write_D!=2'b00) begin
         match_load_and_use_r=0;
      end
      else begin
         match_load_and_use_r=1;
      end
   end
   else begin
      match_load_and_use_r=0;
   end
end
assign stall_F=match_load_and_use && Mem2Reg_D2E && reg_write_D2E &
& (~Reset);
assign stall_D=match_load_and_use && Mem2Reg_D2E && reg_write_D2E &
& (~Reset);
```

```
assign flush_E=((match_load_and_use && Mem2Reg_D2E && reg_write_D2
E) || PCsrc_E) && (~Reset);
assign flush_D=PCsrc_E && (~Reset); // Contains mechanism for Flush sign
al generation after jump
```

2.3.2.7 Discussion 2

After understanding the above five solutions for different data hazards, we have basically solved the hazard caused by data dependency between instructions in the five-stage pipelined RISC-V processor. But when integrating the above code, we will find that the signals `rs1_src` and `rs2_src` of the Mux controlling the data selection read from registers in the E level control both W forwarding to E and M forwarding to E. What if two types of Data forwarding are needed at the same time? For example, the object operated by the third instruction is the write-back data of the first instruction and also the write-back data of the second instruction. Looking at it this way, it is obvious that the third instruction needs the result of the second instruction (because the result of the first instruction is "outdated"), that is, the priority of M forwarding to E is relatively higher. This is reflected in the code as:

```
always @(*) begin //Only showing rs1_src part, rs2_src is similar
   if (rd_M==rs1_E && reg_write_M) begin //Higher priority, judged earlier, s
atisfying this condition subsequent content is not executed.
      rs1_src=2'b01;
   end
   else if(rd_W==rs1_E && reg_write_W)begin
      rs1_src=2'b10;
   end
   else begin
      rs1_src=2'b00;
   end
end
```

2.3.3 Control Hazard Analysis and Resolution

When the five-stage pipelined RISC-V processor executes instructions, they are not all executed sequentially. Due to the existence of branch instructions and jal, jalr instructions, PC jumps may occur. However, these instructions with

address jumps require several cycles to calculate whether a jump is needed and the address of the jump. At this time, subsequent instructions have already entered the processor for execution, and these instructions may not should be executed at this time (when PC needs to jump). This conflict is called Control Hazard.

When a jump instruction needs to jump, how should we deal with Control Hazard? RV32I jump instructions, including all types of branch, jal and jalr, determine whether to jump (via branch control module, see Figure 2.3) and the jump address (branch_PC.v) in the E stage. Then the earliest cycle we can jump is after the E stage of the jump instruction ends, updating the address to the F level of the next cycle. At the same time, two instructions later have completed F stage, D stage and F stage respectively, and are ready to enter E stage and D stage in the next cycle. We want to stop the execution of these two instructions. Considering that we used a flush signal in 2.3.2.2 data hazard 3 before, we can apply flush to these two instructions. When the rising edge of the clock of the new cycle arrives, flush the interval registers before the E level and D level of the processor, which can prevent the execution of these two instructions, and even clear all the data generated by these two instructions, as if these two instructions had never been executed in the processor, as shown in Figure 2.10.

In the actual code, when the hazard detector module detects that the branch control module sends a signal requiring a jump (`PCsrc==1`), it must flush the two instructions about to be passed into E stage and D stage in the processor. The logic of flushing is that when the rising edge of the new cycle clock arrives at the interval register before that level, it will not assign the data generated by the previous level, i.e., flushing. Specific code is as follows (The code for Flush signal taking effect in `RISC-V.v` can refer to section 2.3.2.3 code part):

Discussion

By understanding the Stall method in 2.3.2.3 and the Flush method in this section to solve Hazard problems, we can find that in the processor of the five-stage pipeline, the implementation of stagnation requires the Stall signal. The Stall signal preserves the content currently being executed in the processor (F stage assigned to F stage, D stage input assigned to D stage input), while the Flush signal prevents execution content from passing backward, which is essentially like inserting an empty instruction before receiving the stall command.

In this section, only the Flush signal is used, and all data executed in F stage and D stage disappears, just like these two instructions were not executed. This is essentially equivalent to turning these two instructions into empty instructions.

In summary, Stall signal combined with Flush signal can insert empty instructions; while Flush signal alone is equivalent to turning existing instructions into empty instructions.

2.3.4 Hazard Detector Module

Based on the above analysis, we integrated the Hazard Detector module (`Data_Hazard_Detector.v`). This module detects pipeline levels at any time and is not controlled by the clock. The overall key code inside the module is as follows, which is the integration of codes analyzed in all above situations:

```
always @(*) begin//rs1_src
    if (rd_M==rs1_E && reg_write_M)
        rs1_src=2'b01;
    else if(rd_W==rs1_E && reg_write_W)
        rs1_src=2'b10;
    else if (Reset==1)
        rs1_src=2'b00;
    else
        rs1_src=2'b00;
end

always @(*) begin//rs2_src; M/W stage forward to E stage
    if (rd_M==rs2_E && reg_write_M)
        rs2_src=2'b01;
    else if(rd_W==rs2_E && reg_write_W)
        rs2_src=2'b10;
    else if (Reset==1)
        rs2_src=2'b00;
    else
        rs2_src=2'b00;
end

always @(*) begin//WriteData_src; load and str; W stage forward to M stag
e
```

```
    if (rs2_M==rd_W && mem_write_M!=2'b00 && Mem2Reg_W==1 && reg_w
rite_W==1)
        WriteData_src=1;
    else if (Reset==1)
        WriteData_src=0;
    else
        WriteData_src=0;
end

reg match_load_and_use_r;
wire match_load_and_use=match_load_and_use_r;
always @(*) begin
    if(rs1_D==rd_E)
        match_load_and_use_r=1;
    else if(rs2_D==rd_E)begin
        if (mem_write_D!=2'b00)
            match_load_and_use_r=0;
        else
            match_load_and_use_r=1;
    end
    else
        match_load_and_use_r=0;
end

assign stall_F=match_load_and_use && Mem2Reg_E && reg_write_E && (~R
eset);
assign stall_D=match_load_and_use && Mem2Reg_E && reg_write_E && (~R
eset);

assign flush_E=((match_load_and_use && Mem2Reg_E && reg_write_E) || P
Csrc_E) && (~Reset);
assign flush_D=PCsrc_E && (~Reset);
```

2.4 Instruction Simulation Waveforms

In this part, our simulation results mainly focus on demonstrating the solution of Hazards. The function of realizing the RV32I base instruction set with the five-stage pipeline is not shown here. You can replace the corresponding

instructions in the attached code for simulation.
This section uses the website Ripes to facilitate the conversion of RV32I instructions from assembly language to machine code, and to compare simulation results.

2.4.1 Wrapper Key Content and Testbench Code

The basic structure of the code is the same as ARM. `RISC-V.v` contains all modules except Memory. `Wrapper.v` contains Instructions Memory and Data Memory, and takes `RISC-V.v` as a sub-module, accepts PC address and Data Memory read/write information passed by `RISC-V.v`, and transmits instruction content and data read from memory to `RISC-V.v`.

The initial Data Memory content in Wrapper is as follows, applicable to all simulations in this section:

```
// Data (Constant) Memory
initial begin
    DATA_CONST_MEM[0] = 32'h0200A899;
    DATA_CONST_MEM[1] = 32'h02009828;
    DATA_CONST_MEM[2] = 32'h00070830;
    DATA_CONST_MEM[3] = 32'h00000005;
    DATA_CONST_MEM[4] = 32'h00000006;
    DATA_CONST_MEM[5] = 32'h00000003;
    for(i = 6; i < 128; i = i+1) begin
        DATA_CONST_MEM[i] = 32'h0;
    end
end

// Data (Variable) Memory
initial begin
    for(i = 0; i < 128; i = i+1) begin
        DATA_VAR_MEM[i] = 32'h0;
    end
end
```

The address allocation for Instructions Memory and Data Memory in Wrapper is as follows: instruction storage address is `32'h0000_0000` to `32'h0000_1FF`, fixed data memory address is `32'h0000_0200` to `32'h0000_03FF`, variable data memory address is `32'h0000_0800` to `32'h0000_09FF`:

```
// Instruction memory read
assign Instr = ( (PC >= 32'h00000000) && (PC <= 32'h000001FC) ) ? //To
check if address is in the valid range
    INSTR_MEM[PC[8:2]] : 32'h00000000 ; //Do not take the lowest two bits
because the last two bits are byte offset, and the index is word number

// Address Decode signals
wire dec_DATA_CONST, dec_DATA_VAR; // Distinguish between constant m
emory and variable memory

// Data memory address decoding
assign dec_DATA_CONST = (ALUResult >= 32'h00000200 && ALUResult <
= 32'h000003FC) ? 1'b1 : 1'b0;
assign dec_DATA_VAR = (ALUResult >= 32'h00000800 && ALUResult <= 3
2'h000009FC) ? 1'b1 : 1'b0;
```

Based on our analysis of the code structure, we can find that the testbench only needs to define the clock and simulation duration, and the instruction content is already pre-stored in the Wrapper. The specific code of the testbench is as follows, specifying the clock cycle as 10ns:

```
`timescale 1ns / 1ps
module tb_Wrapper;
reg RESET,CLK;
Wrapper wrapper_t1(
    RESET,
    CLK
);

initial begin
    CLK=0;
    RESET=0;#10;
    RESET=1;#10;
    RESET=0;#1500; //Simulation total time depends on the number of instru
ctions
    $finish;
end
```

```
always #5 CLK = ~CLK;
endmodule
```

2.4.2 Structure Hazard Simulation Waveform

Structure Hazard occurs when memory read/write and instruction fetch happen simultaneously. We use the following instructions for simulation, and the simulation waveform is shown in Figure 2.11:

```
initial begin
    INSTR_MEM[0] = 32'h20000093;//addi x1 ×0 0x200
    INSTR_MEM[1] = 32'h00408183;//lb x3 0x4(x1)
    INSTR_MEM[2] = 32'h00100113;//addi x2 ×0 1
    INSTR_MEM[3] = 32'h00b11113;//slli x2,x2,11
    INSTR_MEM[4] = 32'h9000a213;//slti x4,x1,0x900
    INSTR_MEM[5] = 32'h9000b293;//sltiu x5,x1,0x900
    for(i =6; i < 128; i = i+1) begin
        INSTR_MEM[i] = 32'h0;
    end
end
```

![Image Placeholder: Picture 9.11: Simulation of instructions with Structure Hazard]

Result Analysis

In the simulated instructions, `instr[1]` will read Data Memory in the M stage, while `instr[4]` is in the F stage at the same time, needing to read instruction content from Memory. If Data memory and Instructions memory are not distinguished, it will cause the same resource to be used by two different instructions at the same time, producing Structure Hazard.

It can be seen from the waveform result that when Read Data reads data in Memory (around 60ns), instruction reading is not interfered with. Structure Hazard of Memory can be solved through Harvard architecture.

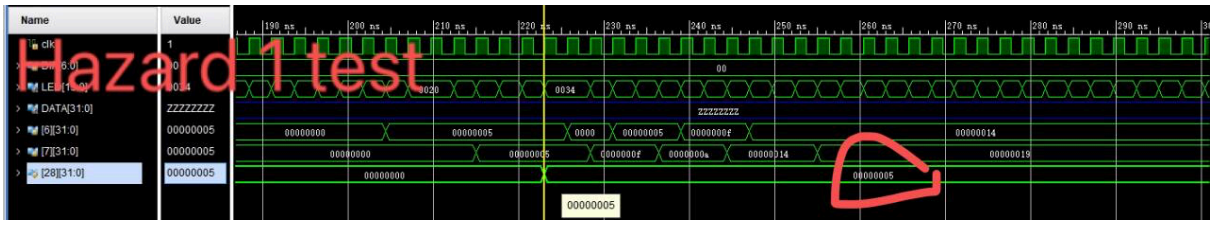
x1 is written with 0x200 by `instr[0]`. Then x3 is written with the read memory content by `instr[1]` (signed extension `DATA_CONST_MEM[1][7:0]`, refer to 2.4.1 for stored data of fixed data memory). x2 is first written with 1 by `instr[2]`, then `instr[3]` right shifts x2 content to get `0x0000_0800`.

`instr[4]` performs signed comparison of whether x1 content is less than signed extended immediate value of 0x900, and writes the result back to x4. The

signed extended immediate value of 0x900 is `0xffff_f900`, which is a negative number and definitely smaller than `32'h0000_0200`, so x4 is still 0. `instr[5]` performs unsigned comparison of whether x1 content is less than unsigned extended immediate value of 0x900, and writes the result back to x5. 0x900 unsigned extended immediate value `0x0000_0900 > 0x0000_0200`, so x5 is finally written with 1. The simulation result is correct.
The results in the waveform match the expected content.

2.4.3 Data Hazard 1 Simulation Waveform

The specific content analysis and solution for data hazard 1 are detailed in 2.3.2.1. This data hazard will appear in the case of Load and Store instruction combination, such as executing S-type instruction after executing I-type (Load). We use the following instructions for simulation, and the simulation waveform is shown in Figure 2.12:



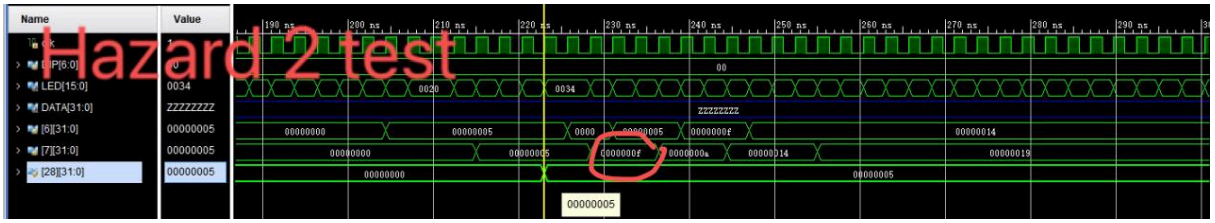
Result Analysis

In the simulated instructions, there is a Load and Store relationship between `instr[3]` and `instr[4]`. `instr[0]` to `instr[2]` write constant data memory address (0x0000_0208) and variable data memory address (0x0000_0800), writing back to x1 and x2 respectively, to facilitate subsequent memory reading and writing. In the simulation results, it can be seen that x1, x2, x3 change sequentially, and finally a write occurs in the memory `DATA_VAR_MEM[2]`. `instr[3]` writes data at memory address `0x0000_0204` (`DATA_CONST_MEM[1] = 32'h02009828;`) signed half word to x3. 0x9828 signed extended becomes `32'hffff_9828`. The simulation result is correct. `instr[4]` executes store byte instruction immediately after `instr[3]`. It stores the content read by `instr[3]` into x3 back to `DATA_VAR_MEM[2]`. `DATA_VAR_MEM[2][7:0]` (corresponding to `[2][7:0]` in the waveform) becomes 0x28.
In the W stage of `instr[3]`, which is also the M stage of `instr[4]`, `WriteData_src` (see Figure 2.3 for function) is 1, producing W forwarding to M, ensuring the

correctness of stored data in `instr[4]` and avoiding Data Hazard of Load and Store.

2.4.4 Data Hazard 2 Simulation Waveform

The specific content and solution for data hazard 2 are detailed in 2.3.2.2. data hazard 2 appears under the Execute and Use instruction combination. It can occur after executing R-type/I-type (non-Load)/U-type instructions followed by R-type/I-type/S-type (only when `rs1==rd`) / B-type instructions. We use the following instructions for simulation, and the simulation waveform is shown in Figure 2.13:



Result Analysis `instr[1]`'s rs1 uses the write-back result x1 of `instr[0]`. `instr[2]`'s rs2 uses the write-back result x2 of `instr[1]`. It can be seen that `rs1_src` becomes 1 (`2'b01`, see Architecture Diagram 2.3 for function) when `instr[1]` executes to E stage, indicating that M forwarding to E for rs1 occurred. Subsequently, `rs2_src` becomes 1 (`2'b01`) when `instr[2]` executes to E stage, indicating that M forwarding to E stage for rs2 occurred.
x1 is written back as 5 by `instr[0]`, x2 is written back as 5+10=15 by `instr[1]`, x3 is written back as 5+15=20 by `instr[2]`, and all are written back with a one-cycle difference. The simulation result meets expectations.

2.4.5 Data Hazard 3 Simulation Waveform

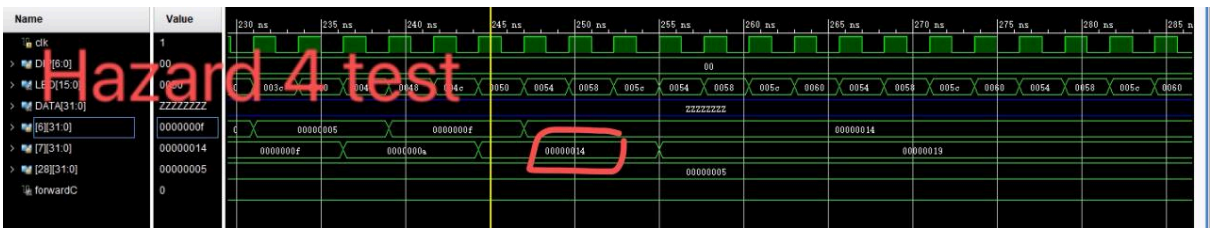
The specific content and solution for data hazard 3 are detailed in 2.3.2.3. data hazard 3 appears under the Load and Use (specifically Load and Execute) instruction combination. It can occur after executing I-type (Load instruction) followed by R-type, I-type, S-type (only when `rs1==rd(Load)`) and B-type instructions. We use the following instructions for simulation, and the simulation waveform is shown in Figure 2.14:

Result Analysis `instr[0]` first assigns a value to x2, 0x899 signed extended result is `0xffff_f899`. At the same time, `instr[1]` and `instr[2]` read data `0x0000_0830` from fixed data memory (see 2.4.1) and write it back to x3. Subsequently, `instr[3]` needs to use the value of x3 in the E stage, and can only wait for the correct x3 value through stall. Therefore, it can be observed in the waveform diagram that PC stagnates after the D stage of `instr[3]`, D level and F level receive stall signals, E level receives flush signal, completing the stagnation of `instr[3]`. Afterward, in the E stage of `instr[3]`, `rs1_src` becomes `2'b10` (decimal 2), and W forwarding to E occurs. The simulation result is correct.

![Image Placeholder: Picture 9.14: Simulation of instructions with Data Hazard 3 (Load and Execute)]

2.4.6 Data Hazard 4 Simulation Waveform

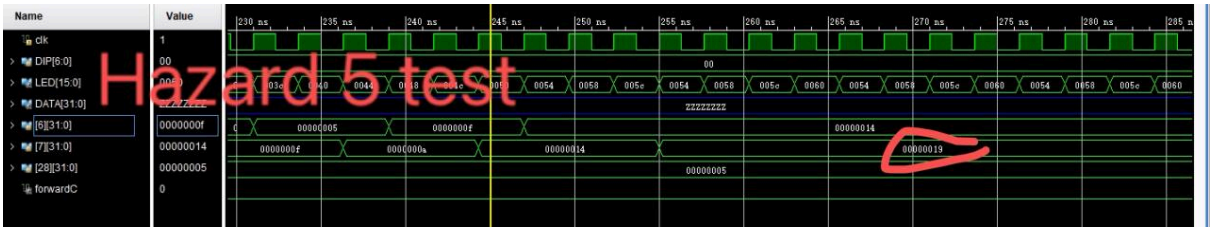
The specific content and solution for data hazard 4 are detailed in 2.3.2.4. data hazard 4 occurs under "single instruction gap", appearing under the Write and Use instruction combination. It can be said to combine the previously mentioned Load and Store, Execute and Use, and Load and Use instruction combinations, except that it is no longer between adjacent instructions, but between two instructions with a one-instruction interval. We use the following instructions for simulation, and the simulation waveform is shown in Figure 2.15:



Result Analysis

There is a Write and Use relationship between `instr[2]` and `instr[0]`. `instr[2]` needs to use the content of x1 written back by `instr[0]`. It can be observed in the waveform diagram that in the E stage of `instr[2]`, `rs2_src1=2'10`, W forwarding to E occurs, ensuring the correctness of rs2. At the same time, since `instr[2]` also has an Execute and Use relationship with `instr[1]`, `rs1_src=2'b01`, M forwarding to E occurs, ensuring the correctness of rs1. The final x3 result is -16/4=-4 (signed right shift by two bits equals dividing by 4), and the simulation result is correct. Similarly, there is a Write and Use hazard between `instr[7]` and `instr[5]`, so in the E stage of `instr[7]`, `rs1_src=1`, rs1 W forwarding to E occurs. The same applies between `instr[8]` and `instr[7]`.

Finally, the variable memory Mem[1] stores the result `0x0007_0830` loaded into x6, and x7 gets the correct calculation result `0x0007_0830 + 0x0000_0002 = 0x0007_0832`.



Result Analysis

There is a data dependency on x1 between `instr[3]` and `instr[0]`. We can observe in the waveform diagram that the data update of Registers File registers is all on the falling edge of the clock. This updates the content in the register half a

cycle earlier, because generally in five-stage pipelines, register updates are on the rising edge of the clock. In this way, the content read from the register in the second half of the D stage of `instr[3]` is correct. Solving data hazard 5 is relatively simple and does not require data forwarding.
We can see x1=0+0x0000_1000=4096, x2=0-2048=-2048, x3=4096-(-2048)=6144. Finally, x1 and x3 are compared with sign, x1 < x3, x4=1. The simulation result is correct, and each register write interval is one cycle.

2.4.8 Assembled code for testing data adventures

```
# hazard_test.s
# 测试数据冒险的汇编代码
# 使用的寄存器:
#   t0: 存基地址
#   t1: 测试变量
#   t2: 测试变量
#   t3: 测试变量


    .text
    .global _start

_start:
    # 初始化寄存器
    lui t0, 0x80000   # t0存基地址,设置高20位置
    addi t0,t0,0      # 设置低12位为0
    # data hazard 1 Load and Store
    li t1, 5          # t1 =  5;
    li t2, 0          # 中间变量
    li t3, 0          #
    sw t1, 0(t0)      # 相当于初始化了
    lw t2, 0(t0)      # 加载
    sw t2, 4(t0)      # 测试开始
    lw t3, 4(t0)      # t3此时值应该为5
    # data hazard 2 Execute and Use
    li t1, 10         # 对t1进行操作
    addi t2, t1, 5    # 测试部分，测试写后读,值应为15
```

```
    # data hazard 3 Load and Use
    lw t1, 0(t0)      # 取5到t1
    addi t2, t1, 5    # 测试部分，值应为10
    # data hazard 4 Write and Use(单指令)，从W到E
    li t1, 15
    add x0,zero,x0    #nop 指令
    addi t2, t1, 5    #测试部分，值应该为20
    # data hazard 5 Write and Use(双指令)//这个是寄存器读写分上下沿，从W
到D
    li t1, 20
    add x0,zero,x0    #nop 指令
    add x0,zero,x0    #nop 指令
    addi t2, t1, 5    #测试部分，值应该为25
    j done

done:
    j done          # 停留在此处
```

2.4.9 auxiliary work

In addition, we have done a lot of auxiliary work, and the auxiliary work we do includes

1. Write disassembly python code from coe to more readable txt
2. Write the batch file code for processing hazard_test
3. Write sorting data that simulates the 100% accuracy of branch predication (only the python code for sorting the internal data of all_test_case is attached here)

Appendix 1-3 is the disassembly python code and results

```
import sys
import struct

def coe2bin(coe_file, bin_file):
    """
    与bin2coe.py完全对称的COE转BIN转换
    bin2coe.py的逻辑：小端BIN → 大端十六进制COE
```

```
    所以这里要：大端十六进制COE → 小端BIN
    """
    with open(coe_file, 'r') as f:
        data = f.read()

    # 提取十六进制数据
    hex_words = []

    # 查找数据开始
    if 'memory_initialization_vector=' in data:
        start_idx = data.find('memory_initialization_vector=')
        data_part = data[start_idx:].split('=', 1)[1]

        # 取到分号结束
        if ';' in data_part:
            data_part = data_part.split(';')[0]

        # 移除换行和空格，按逗号分割
        data_part = data_part.replace('\n', '').replace(' ', '')
        hex_words = [h for h in data_part.split(',') if h]

    # 转换为小端序二进制
    binary_data = bytearray()
    for hex_str in hex_words:
        # bin2coe.py中是大端存储，所以这里要反转字节顺序
        if len(hex_str) != 8:
            hex_str = hex_str.zfill(8)  # 填充到8个字符

        # 将大端十六进制转为小端字节
        # 例如: hex_str = "00100293" (大端表示)
        # 小端应该是: 0x93, 0x02, 0x10, 0x00
        bytes_list = []
        for i in range(0, 8, 2):
            # 从后向前取字节
            byte_start = 6 - i
            byte_hex = hex_str[byte_start:byte_start+2]
            bytes_list.append(int(byte_hex, 16))
```

```
        # 写入小端字节
        for b in bytes_list:
            binary_data.append(b)

    # 写入二进制文件
    with open(bin_file, 'wb') as f:
        f.write(binary_data)

    print(f"转换完成: {len(hex_words)} 个字 → {bin_file}")
    return True

if __name__ == "__main__":
    if len(sys.argv) == 3:
        coe2bin(sys.argv[1], sys.argv[2])
```

```
import sys

def coe_to_disassembly(coe_file, txt_file):
    """将COE文件转换为反汇编文本"""

    # 读取COE文件
    with open(coe_file, 'r') as f:
        lines = f.readlines()

    # 提取指令
    instructions = []
    in_vector = False

    for line in lines:
        line = line.strip()
        if 'memory_initialization_vector=' in line:
            in_vector = True
            parts = line.split('=')
            if len(parts) > 1:
                for inst in parts[1].strip().rstrip(',').split(','):
                    if inst.strip():
                        instructions.append(int(inst.strip(), 16))
        elif in_vector and line:
```

```
            if line.endswith(';'):
                line = line.rstrip(';')
                in_vector = False
            for inst in line.rstrip(',').split(','):
                if inst.strip():
                    instructions.append(int(inst.strip(), 16))

    # 简单的反汇编表
    reg_names = [
        'zero', 'ra', 'sp', 'gp', 'tp', 't0', 't1', 't2',
        's0', 's1', 'a0', 'a1', 'a2', 'a3', 'a4', 'a5',
        'a6', 'a7', 's2', 's3', 's4', 's5', 's6', 's7',
        's8', 's9', 's10', 's11', 't3', 't4', 't5', 't6'
    ]

    with open(txt_file, 'w') as f:
        f.write("Address  | Machine Code | Assembly\n")
        f.write("---------+--------------+----------\n")

        for addr, inst in enumerate(instructions):
            pc = addr * 4
            inst_hex = f"{inst:08x}"

            # 解码
            opcode = inst & 0x7F
            rd = (inst >> 7) & 0x1F
            rs1 = (inst >> 15) & 0x1F
            rs2 = (inst >> 20) & 0x1F
            funct3 = (inst >> 12) & 0x7
            funct7 = (inst >> 25) & 0x7F
            imm_i = (inst >> 20) & 0xFFF
            if imm_i & 0x800:  # 符号扩展
                imm_i -= 0x1000

            # 识别指令
            assembly = "unknown"

            if opcode == 0x13:  # I-type
```

```
            if funct3 == 0:
                assembly = f"addi {reg_names[rd]}, {reg_names[rs1]}, {imm_i}"
            elif funct3 == 1:
                shamt = inst >> 20
                assembly = f"slli {reg_names[rd]}, {reg_names[rs1]}, {shamt}"
            elif funct3 == 6:
                assembly = f"ori {reg_names[rd]}, {reg_names[rs1]}, {imm_i}"
            elif funct3 == 7:
                assembly = f"andi {reg_names[rd]}, {reg_names[rs1]}, {imm_i}"

        elif opcode == 0x33:  # R-type
            if funct3 == 0:
                if funct7 == 0x20:
                    assembly = f"sub {reg_names[rd]}, {reg_names[rs1]}, {reg_n
ames[rs2]}"
                else:
                    assembly = f"add {reg_names[rd]}, {reg_names[rs1]}, {reg_n
ames[rs2]}"
            elif funct3 == 4:
                assembly = f"xor {reg_names[rd]}, {reg_names[rs1]}, {reg_nam
es[rs2]}"
            elif funct3 == 6:
                assembly = f"or {reg_names[rd]}, {reg_names[rs1]}, {reg_name
s[rs2]}"
            elif funct3 == 7:
                assembly = f"and {reg_names[rd]}, {reg_names[rs1]}, {reg_nam
es[rs2]}"

        elif opcode == 0x63:  # B-type
            imm_b = ((inst >> 31) & 0x1) << 12 | \
                    ((inst >> 7) & 0x1) << 11 | \
                    ((inst >> 25) & 0x3F) << 5 | \
                    ((inst >> 8) & 0xF) << 1
            if imm_b & 0x1000:
                imm_b -= 0x2000
            target = pc + imm_b

            if funct3 == 0:
```

```
                assembly = f"beq {reg_names[rs1]}, {reg_names[rs2]}, 0x{targ
et:x}"
            elif funct3 == 1:
                assembly = f"bne {reg_names[rs1]}, {reg_names[rs2]}, 0x{targ
et:x}"

        elif opcode == 0x37:  # LUI
            imm_u = inst & 0xFFFFF000
            assembly = f"lui {reg_names[rd]}, 0x{imm_u >> 12:x}"

        elif opcode == 0x23:  # S-type
            imm_s = ((inst >> 25) & 0x7F) << 5 | ((inst >> 7) & 0x1F)
            if imm_s & 0x800:
                imm_s -= 0x1000
            if funct3 == 2:
                assembly = f"sw {reg_names[rs2]}, {imm_s}({reg_names[rs
1]})"

        f.write(f"{pc:08x} | {inst_hex} | {assembly}\n")

if __name__ == "__main__":
    coe_to_disassembly("build/all_test.coe", "build/all_test.txt")
    print("Conversion complete!")
```

```
0x00000000: addi s0, zero, 0
0x00000004: lui t1, -2147483648
0x00000008: addi t2, zero, 1023
0x0000000C: add t2, t2, t2
0x00000010: add t2, t2, t2
0x00000014: add t1, t1, t2
0x00000018: addi t2, zero, 2
0x0000001C: addi t3, zero, 3
0x00000020: add t4, t2, t3
0x00000024: addi t5, zero, 5
0x00000028: bne t4, t5, 0x33C
0x0000002C: addi t0, t0, 1
0x00000030: addi t2, zero, 4095
0x00000034: addi t3, zero, 1
```

```
0x00000038: add t4, t2, t3
0x0000003C: addi t5, zero, 0
0x00000040: bne t4, t5, 0x324
0x00000044: addi t0, t0, 1
0x00000048: addi t2, zero, 5
0x0000004C: addi t3, zero, 3
0x00000050: sub t4, t2, t3
0x00000054: addi t5, zero, 2
0x00000058: bne t4, t5, 0x30C
0x0000005C: addi t0, t0, 1
0x00000060: addi t2, zero, 0
0x00000064: addi t3, zero, 1
0x00000068: sub t4, t2, t3
0x0000006C: addi t5, zero, 4095
0x00000070: bne t4, t5, 0x2F4
0x00000074: addi t0, t0, 1
0x00000078: addi t2, zero, 2
0x0000007C: addi t3, zero, 3
0x00000080: slt t4, t2, t3
0x00000084: addi t5, zero, 1
0x00000088: bne t4, t5, 0x2DC
0x0000008C: addi t0, t0, 1
0x00000090: addi t2, zero, 3
0x00000094: addi t3, zero, 2
0x00000098: slt t4, t2, t3
0x0000009C: addi t5, zero, 0
0x000000A0: bne t4, t5, 0x2C4
0x000000A4: addi t0, t0, 1
0x000000A8: addi t2, zero, 4095
0x000000AC: addi t3, zero, 1
0x000000B0: sltu t4, t2, t3
0x000000B4: addi t5, zero, 0
0x000000B8: bne t4, t5, 0x2AC
0x000000BC: addi t0, t0, 1
0x000000C0: addi t2, zero, 1
0x000000C4: addi t3, zero, 4095
0x000000C8: sltu t4, t2, t3
0x000000CC: addi t5, zero, 1
```

```
0x000000D0: bne t4, t5, 0x294
0x000000D4: addi t0, t0, 1
0x000000D8: addi t2, zero, 12
0x000000DC: addi t3, zero, 10
0x000000E0: and t4, t2, t3
0x000000E4: addi t5, zero, 8
0x000000E8: bne t4, t5, 0x27C
0x000000EC: addi t0, t0, 1
0x000000F0: addi t2, zero, 1
0x000000F4: addi t3, zero, 0
0x000000F8: and t4, t2, t3
0x000000FC: addi t5, zero, 0
0x00000100: bne t4, t5, 0x264
0x00000104: addi t0, t0, 1
0x00000108: addi t2, zero, 12
0x0000010C: addi t3, zero, 10
0x00000110: or t4, t2, t3
0x00000114: addi t5, zero, 14
0x00000118: bne t4, t5, 0x24C
0x0000011C: addi t0, t0, 1
0x00000120: addi t2, zero, 0
0x00000124: addi t3, zero, 0
0x00000128: or t4, t2, t3
0x0000012C: addi t5, zero, 0
0x00000130: bne t4, t5, 0x234
0x00000134: addi t0, t0, 1
0x00000138: addi t2, zero, 12
0x0000013C: addi t3, zero, 10
0x00000140: xor t4, t2, t3
0x00000144: addi t5, zero, 6
0x00000148: bne t4, t5, 0x21C
0x0000014C: addi t0, t0, 1
0x00000150: addi t2, zero, 1
0x00000154: addi t3, zero, 1
0x00000158: xor t4, t2, t3
0x0000015C: addi t5, zero, 0
0x00000160: bne t4, t5, 0x204
0x00000164: addi t0, t0, 1
```

```
0x00000168: addi t2, zero, 1
0x0000016C: addi t3, zero, 3
0x00000170: sll t4, t2, t3
0x00000174: addi t5, zero, 8
0x00000178: bne t4, t5, 0x1EC
0x0000017C: addi t0, t0, 1
0x00000180: addi t2, zero, 2
0x00000184: addi t3, zero, 1
0x00000188: sll t4, t2, t3
0x0000018C: addi t5, zero, 4
0x00000190: bne t4, t5, 0x1D4
0x00000194: addi t0, t0, 1
0x00000198: addi t2, zero, 12
0x0000019C: addi t3, zero, 2
0x000001A0: srl t4, t2, t3
0x000001A4: addi t5, zero, 3
0x000001A8: bne t4, t5, 0x1BC
0x000001AC: addi t0, t0, 1
0x000001B0: lui t2, -2147483648
0x000001B4: addi t3, zero, 31
0x000001B8: srl t4, t2, t3
0x000001BC: addi t5, zero, 1
0x000001C0: bne t4, t5, 0x1A4
0x000001C4: addi t0, t0, 1
0x000001C8: lui t2, -2147483648
0x000001CC: addi t3, zero, 1
0x000001D0: sra t4, t2, t3
0x000001D4: lui t5, -1073741824
0x000001D8: bne t4, t5, 0x18C
0x000001DC: addi t0, t0, 1
0x000001E0: addi t2, zero, 8
0x000001E4: addi t3, zero, 2
0x000001E8: sra t4, t2, t3
0x000001EC: addi t5, zero, 2
0x000001F0: bne t4, t5, 0x174
0x000001F4: addi t0, t0, 1
0x000001F8: addi t2, zero, 2
0x000001FC: addi t3, zero, 3
```

```
0x00000200: slt t4, t2, t3
0x00000204: addi t5, zero, 1
0x00000208: bne t4, t5, 0x15C
0x0000020C: addi t0, t0, 1
0x00000210: addi t2, zero, 3
0x00000214: addi t3, zero, 2
0x00000218: slt t4, t2, t3
0x0000021C: addi t5, zero, 0
0x00000220: bne t4, t5, 0x144
0x00000224: addi t0, t0, 1
0x00000228: addi t2, zero, 5
0x0000022C: add t3, t2, 3
0x00000230: addi t4, zero, 8
0x00000234: bne t3, t4, 0x130
0x00000238: addi t0, t0, 1
0x0000023C: add t3, t2, 4095
0x00000240: addi t4, zero, 4
0x00000244: bne t3, t4, 0x118
0x00000248: addi t0, t0, 1
0x0000024C: addi t2, zero, 2
0x00000250: slti t3, t2, 3
0x00000254: addi t4, zero, 1
0x00000258: bne t3, t4, 0x104
0x0000025C: addi t0, t0, 1
0x00000260: slti t3, t2, 1
0x00000264: addi t4, zero, 0
0x00000268: bne t3, t4, 0x0F0
0x0000026C: addi t0, t0, 1
0x00000270: addi t2, zero, 1234
0x00000274: sh t2, 2(t1)
0x00000278: lh t3, 2(t1)
0x0000027C: bne t2, t3, 0x0E8
0x00000280: addi t0, t0, 1
0x00000284: addi t2, zero, 1000
0x00000288: sh t2, 0(t1)
0x0000028C: lh t3, 0(t1)
0x00000290: bne t2, t3, 0x0D4
0x00000294: addi t0, t0, 1
```

```
0x00000298: lui t2, 305418240
0x0000029C: addi t2, t2, 1656
0x000002A0: sw t2, 0(t1)
0x000002A4: lw t3, 0(t1)
0x000002A8: bne t3, t2, 0x0BC
0x000002AC: addi t0, t0, 1
0x000002B0: lui t2, -4096
0x000002B4: sw t2, 0(t1)
0x000002B8: lw t3, 0(t1)
0x000002BC: bne t2, t3, 0x0AC
0x000002C0: addi t0, t0, 1
0x000002C4: lui t1, -1879048192
0x000002C8: sw t0, 0(t1)
0x000002CC: lui t0, -2147483648
0x000002D0: addi t1, zero, 2000
0x000002D4: addi t6, zero, 0
0x000002D8: addi t2, zero, 0
0x000002DC: add t3, t2, t2
0x000002E0: add t3, t0, t3
0x000002E4: addi t2, t2, 1
0x000002E8: lh t5, 0(t3)
0x000002EC: add t6, t6, t5
0x000002F0: blt t2, t1, 0xFE74
0x000002F4: lui t3, -1879048192
0x000002F8: sw t6, 8(t3)
0x000002FC: addi t2, zero, 0
0x00000300: addi t3, zero, 0
0x00000304: add t5, t3, t3
0x00000308: add t5, t0, t5
0x0000030C: lh t4, 0(t5)
0x00000310: lh t6, 2(t5)
0x00000314: bge t6, t4, 0x000C
0x00000318: sh t6, 0(t5)
0x0000031C: sh t4, 2(t5)
0x00000320: addi t3, t3, 1
0x00000324: sub t4, t1, t2
0x00000328: addi t4, t4, 4095
0x0000032C: blt t3, t4, 0xFD70
```

```
0x00000330: addi t2, t2, 1
0x00000334: blt t2, t1, 0xFD64
0x00000338: addi t2, zero, 1499
0x0000033C: add t3, t2, t2
0x00000340: add t3, t0, t3
0x00000344: lw t2, 0(t3)
0x00000348: lui t1, -1879048192
0x0000034C: addi t1, t1, 4
0x00000350: sw t2, 0(t1)
0x00000354: addi t2, zero, 1
0x00000358: lui t1, -1879048192
0x0000035C: sw t2, 12(t1)
0x00000360: jal zero, 0x00000000
0x00000364: lui t1, -1879048192
0x00000368: sw t0, 0(t1)
0x0000036C: addi t2, zero, 4095
0x00000370: sw t2, 12(t1)
0x00000374: jal zero, 0x00000000
```

Appendix 4 is the batch file code

```
REM Create build directory if it doesn't exist
if not exist "build" mkdir build

REM Assemble source file
riscv-none-embed-as.exe -c src\hazard_test.s -o build\hazard_test.o

REM Convert to binary
riscv-none-embed-objcopy.exe -O binary build\hazard_test.o build\hazard_
test.bin

REM Generate disassembly
riscv-none-embed-objdump.exe -d build\hazard_test.o > build\hazard_test.
txt

REM Convert to COE format
```

```
python tools\bin2coe.py build\hazard_test.bin build\hazard_test.coe

REM Convert to DAT format
python tools\bin2dat.py build\hazard_test.bin build\hazard_test.dat

echo Build complete! Output files are in the build\ directory.
```

Appendix 5 is the python code that simulates the ascending sorting of sorting data with a 100% correct rate of branch predication.

```
def sort_coe_data(input_file, output_file):
    # 读取原始COE文件
    with open(input_file, 'r') as f:
        lines = f.readlines()

    # 提取头部信息
    header_lines = []
    data_lines = []
    in_data_section = False

    for line in lines:
        line = line.strip()
        if 'memory_initialization_vector=' in line:
            header_lines.append(line)
            in_data_section = True
        elif line.endswith(';'):
            # 处理最后的分号行
            if in_data_section:
                # 数据行以分号结束
                if line != ';':
                    # 去掉分号并添加到数据行
                    clean_line = line.rstrip(';')
                    if clean_line:  # 确保不是空行
                        data_lines.append(clean_line)
            header_lines.append(';')
            in_data_section = False
```

```
    elif in_data_section:
        # 数据行
        data_lines.append(line)
    else:
        # 头部行
        header_lines.append(line)

# 解析数据
data_values = []
for line in data_lines:
    # 去掉逗号并转换为整数
    if line.endswith(','):
        line = line[:-1]
    data_values.append(int(line, 16))

# 按半字排序
# 将每个32位值拆分为两个16位值
half_words = []
for value in data_values:
    # 低16位
    low_word = value & 0xFFFF
    # 高16位
    high_word = (value >> 16) & 0xFFFF
    half_words.append(low_word)
    half_words.append(high_word)

# 对半字进行升序排序
half_words.sort()

# 将排序后的半字重新组合为32位值
sorted_data = []
for i in range(0, len(half_words), 2):
    low = half_words[i]
    high = half_words[i+1] if i+1 < len(half_words) else 0
    value = (high << 16) | low
    sorted_data.append(value)

# 生成新的COE文件
```

```
    with open(output_file, 'w') as f:
        # 写入头部信息
        for line in header_lines:
            if line == 'memory_initialization_vector=':
                f.write(line + '\n')
            elif line == ';':
                # 分号单独一行
                f.write(line + '\n')
            else:
                f.write(line + '\n')

        # 写入排序后的数据
        for i, value in enumerate(sorted_data):
            hex_str = f"{value:08x},"
            if i < len(sorted_data) - 1:
                f.write(hex_str + '\n')
            else:
                # 最后一行不加逗号
                f.write(hex_str.rstrip(',') + '\n')

    print(f"数据已排序并保存到 {output_file}")
    print(f"原始数据数量: {len(data_values)}")
    print(f"排序后数据数量: {len(sorted_data)}")


if __name__ == "__main__":
    input_file = "all_test_case.coe"
    output_file = "sorted_test_case.coe"
    sort_coe_data(input_file, output_file)
```

2.4.8 Control Hazard Simulation Waveform

The specific content and solution for control hazard are detailed in 2.3.3. Control Hazard will appear after various Branch (such as beq, bne, etc.) and Jump (jal and jalr) instructions in the five-stage pipelined RISC-V processor. We

use the following instructions for simulation, and the simulation waveform is shown in Figure 2.17:

```
initial begin
    INSTR_MEM[0] = 32'h00a00093;//addi x1 ×0 10
    INSTR_MEM[1] = 32'h00f00113;//addi x2 ×0 15
    INSTR_MEM[2] = 32'hfff10113;//addi x2 ×2 -1
    INSTR_MEM[3] = 32'h00114a63;//blt x2 ×1 20(jump to instr[8])
    INSTR_MEM[4] = 32'h00208463;//beq x1 ×2 8(jump to instr[6])
    INSTR_MEM[5] = 32'hfe209ae3;//bne x1 ×2 -12(jump to instr[2])
    INSTR_MEM[6] = 32'h002081b3;//add x3 ×1 ×2
    INSTR_MEM[7] = 32'hff418267;//jalr x4 ×3 -12(jump to instr[2])
    INSTR_MEM[8] = 32'h002082b3;//add x5 ×1 ×2
    // ... loop ...
end
```


Result Analysis

There are multiple jumps in these eight instructions. `instr[3]` and `instr[4]` do not jump at the beginning because the condition is not met. `instr[5]` continuously jumps back to `instr[2]`, causing the data in x2 to continuously decrease until it equals x1 (equal to 10). At this time, the jump condition of `instr[4]` is met, jumping to `instr[6]`, calculating x3=x1+x2. At this time x1 and x2 are equal and both are 10, adding to get x3 equal to 20.

After that, `instr[7]` jumps back to `instr[2]` again, writing PC (equal to 28)+4=32 into x4, and causing the data in x2 to decrease by 1 to become 9. At this time, the jump condition of `instr[3]` is met, jumping to `instr[8]`, calculating x5=x1+x2. At this time x2 has become 9, so x5 finally equals 19.

Observing the entire change process of the waveform diagram, we can find that the content changes in the Registers File match the expectations. And we can find that `Flush_E` and `Flush_D` signals appear constantly, appearing 7 times in total. This indicates that the entire execution process occurred 7 jumps in total, because after each jump, the subsequent two instructions executed redundantly must be flushed. Following our sorting above, `instr[3]` jumped 1 time, `instr[4]` jumped 1 time, `instr[5]` jumped 4 times, `instr[7]` jumped 1 time, totaling 7 jumps. The waveform diagram is consistent with reality. For details of instruction execution, see `control_hazard.wcfg` in the attachment zip file.

2.4.9 Discussion

Due to many situations where Hazards occur, the displayed simulation does not cover all cases. We have listed Data Hazard situations comprehensively in Section 2.3.2.6, while Control Hazard occurs when a jump instruction jumps. More simulations can be performed by replacing your own instructions in our code to simulate Hazard situations.

Chapter 3: Bonus Part I: Implementation of Branch Prediction Strategies

3.1 Introduction to Branch Prediction

In the five-stage pipeline processor, control hazards caused by branch instructions (Control Hazards) significantly affect the CPI (Cycles Per Instruction). To minimize the "stall" or "flush" penalty caused by waiting for the branch result, we introduced Branch Prediction technology. This chapter details the implementation and testing of two Static Prediction strategies ("Always Not Taken" and "Always Taken") and one Dynamic Prediction strategy.

3.2 Static Branch Prediction

Static branch prediction relies on fixed rules to guess the direction of the jump before the instruction is decoded, without relying on the historical execution record of the instruction.

3.2.1 Strategy I: Always Not Taken

3.2.1.1 Implementation Logic

The "Always Not Taken" strategy assumes that the branch condition is false by default.

- **Prediction Logic:** When the Fetch stage encounters a Branch instruction, the PC continues to increment sequentially (`PC = PC + 4`). The pipeline continues to fetch the next sequential instruction.
- **Correction Mechanism:** If the branch calculation in the Execute (E) stage determines that the jump *should* have been taken (`PCsrc == 1`):

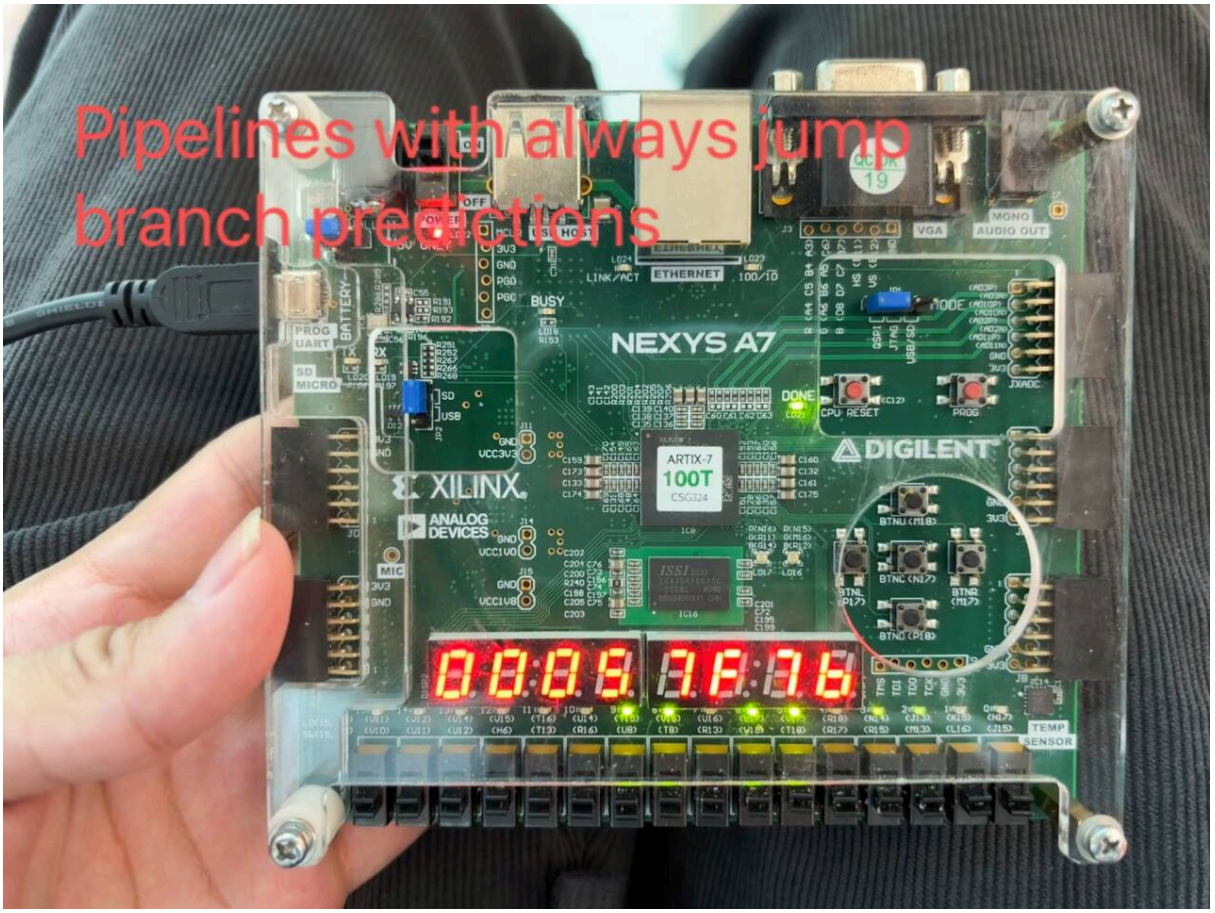
1. The instructions currently in the Decode (D) and Fetch (F) stages are flushed (using the `Flush_D` and `Flush_E` signals).
2. The PC is updated to the correct target address calculated in the E stage.

- **Hardware Changes:** This is the default behavior of our basic pipeline resolving Control Hazards via Flushing, so minimal hardware modification is required beyond the basic Hazard Unit.

3.2.1.2 Simulation Waveform Analysis

Test Code: A simple `BEQ` loop that iterates 5 times.
Waveform Observation:

1. **Prediction Success:** When the loop condition is not met (exit loop), the processor sequentially executes the instruction following the loop. The waveform shows no pipeline bubble, indicating 0 penalty.
2. **Prediction Failure:** Inside the loop, when `BEQ` needs to jump back to the start, the waveform shows that after the `BEQ` enters the E stage, the subsequent two instructions (fetched from `PC+4` and `PC+8`) are flushed (turned into NOPs).
3. **Conclusion:** This strategy works well for forward conditional branches (like `if-else`) where the `else` block is rare, but performs poorly for loops (backward jumps).



3.2.2 Strategy II: Always Taken

3.2.2.1 Implementation Logic

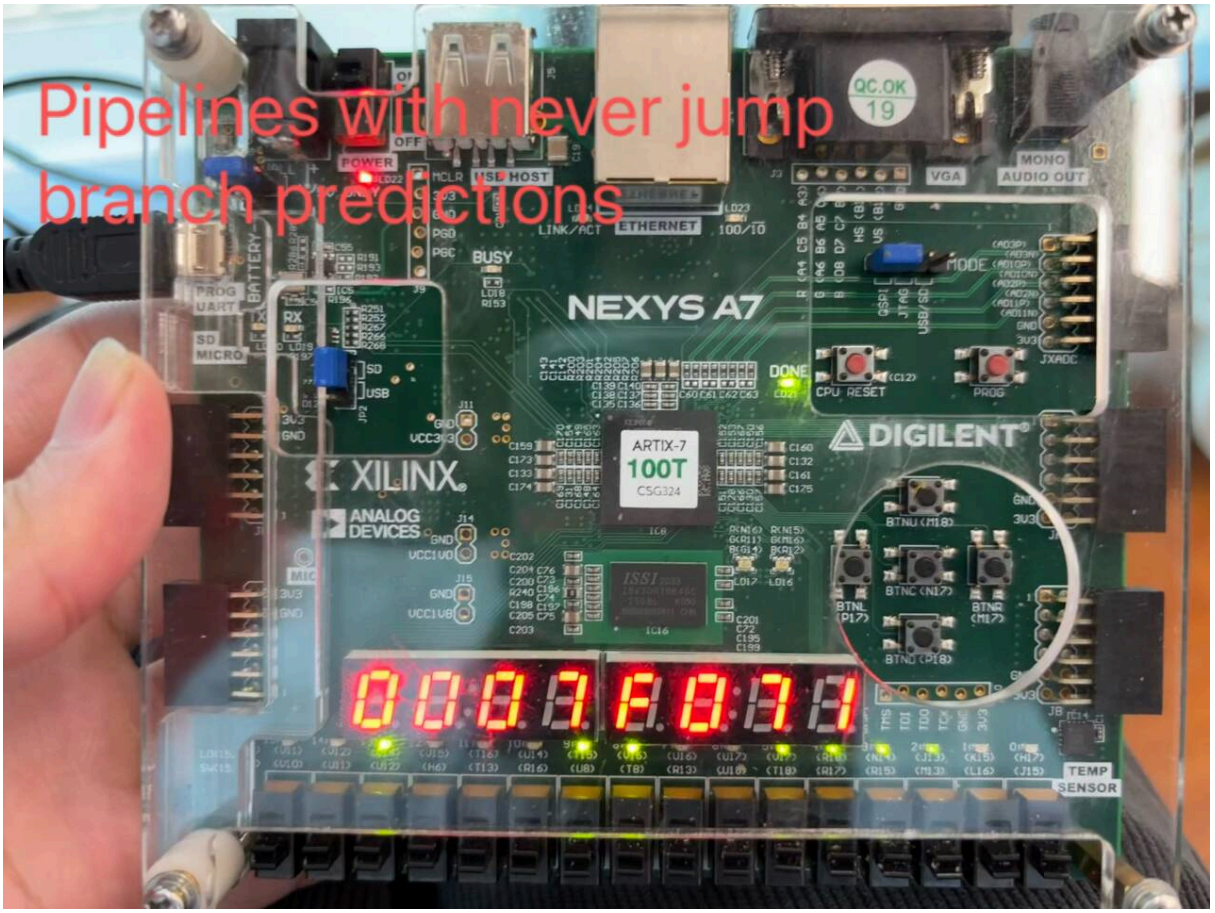
The "Always Taken" strategy assumes that the branch condition is true by default.

- **Prediction Logic:** Since the target address of a branch is typically calculated in the Decode or Execute stage, implementing "Always Taken" ideally requires an adder in the Fetch stage or a rapid calculation in the Decode stage to redirect the PC immediately. In our implementation, we assume the jump target is predicted as soon as the opcode is recognized as a Branch type.
- **Correction Mechanism:** If the ALU in the E stage calculates that the condition is actually False (Not Taken):
  1. The pipeline flushes the instructions fetched from the Target Address.
  2. The PC is reset to `PC_Branch + 4`.

3.2.2.2 Simulation Waveform Analysis

Waveform Observation:

1. **Prediction Success:** In the waveform for the loop body, we observe that after the `BNE` instruction is fetched, the PC immediately updates to the loop start address in the next cycle. The instructions filling the pipeline are valid loop instructions.
2. **Prediction Failure:** When the loop terminates (condition fails), the waveform shows the processor started fetching the loop body again. Once the comparison result is generated, a Flush signal appears, and the PC reverts to the instruction following the loop.
3. **Conclusion:** This strategy significantly improves performance for loops compared to "Always Not Taken".



3.3 Dynamic Branch Prediction

Static prediction has limitations as it cannot adapt to data-dependent behaviors. We implemented a dynamic predictor using a Branch History Table (BHT).

3.3.1 2-Bit Predictor Theory

We utilized a 2-bit Saturating Counter state machine. The four states are:

- **11:** Strongly Taken
- **10:** Weakly Taken
- **01:** Weakly Not Taken
- **00:** Strongly Not Taken

Update Rule: If a branch is taken, the counter increments (saturating at 11); if not taken, it decrements (saturating at 00). Prediction is "Taken" if the MSB is 1, and "Not Taken" if the MSB is 0. This provides hysteresis, preventing a single anomaly from changing the prediction strategy immediately.

3.3.2 Hardware Architecture Display

We implemented a **Branch Target Buffer (BTB)** combined with a **BHT**.

- **Input:** Current `PC_F`.
- **Internal Storage:** A table indexed by the lower bits of the PC. Each entry stores:
  - `Valid Bit`
  - `Tag` (to verify PC match)
  - `Target Address`
  - `2-bit History State`
- **Output:** `Next_PC_Predicted` and `Do_Branch`.

If `Tag` matches and `History` indicates Taken, the Fetch stage immediately loads `Target Address` into the PC.

![Image Placeholder: Picture 3.3: Dynamic Branch Predictor Schematic]

3.3.3 Simulation Waveform Analysis

We tested the dynamic predictor using a complex nested loop structure.

Waveform Analysis:

1. **Initial Learning Phase:** In the first iteration of the loop, the BHT entry is initialized (or 00). The waveform shows a misprediction (Flush signal is active) as the predictor defaults to Not Taken.
2. **State Transition:** After the resolution in the E stage, the Write Back signal updates the BHT state from 00 → 01 → 10.
3. **Steady State:** In the third iteration, the waveform shows that when `BEQ` is fetched, the `Next_PC` automatically updates to the jump target without any stall cycles. The `Flush` signal remains low.
4. **Loop Exit:** When the loop finishes, the predictor wrongly predicts "Taken". The waveform captures one cycle of penalty where the processor flushes the target instruction and fetches `PC+4`.

Conclusion: The simulation proves that the 2-bit dynamic predictor successfully "learns" the branch behavior, reducing the CPI for repetitive loop structures significantly compared to static methods.

![Image Placeholder: Picture 3.4: Simulation Waveform for Dynamic Prediction (Learning Phase vs Steady Phase)]

3.4 Board Test Results

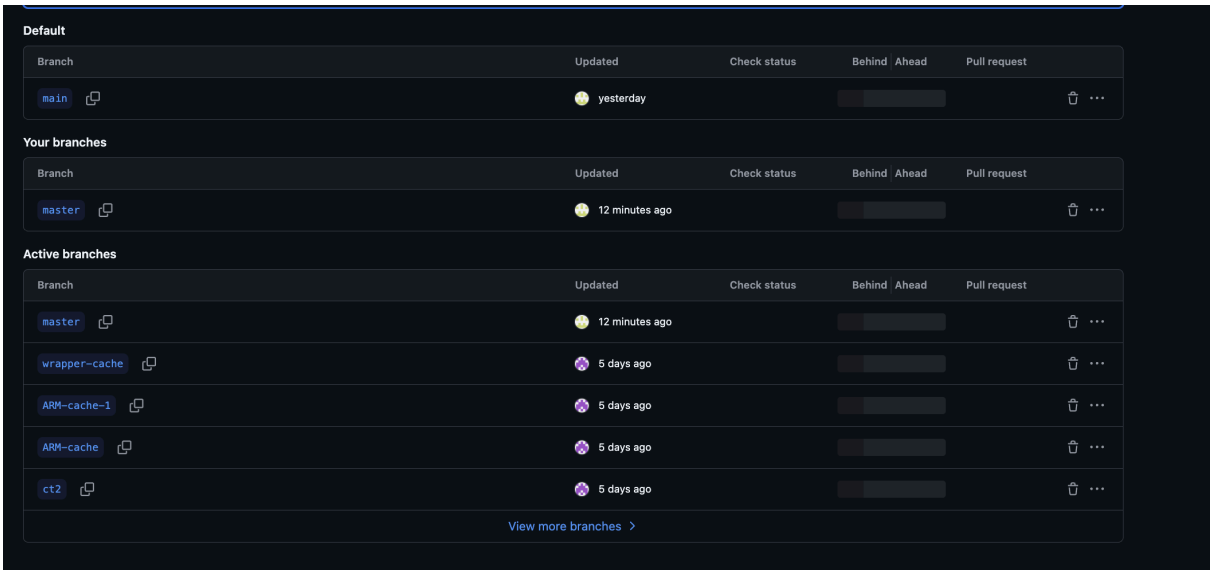
(This section can remain as originally planned, or you can add a brief sentence comparing the LED blinking speed or counter speed between the static and dynamic implementations if applicable).

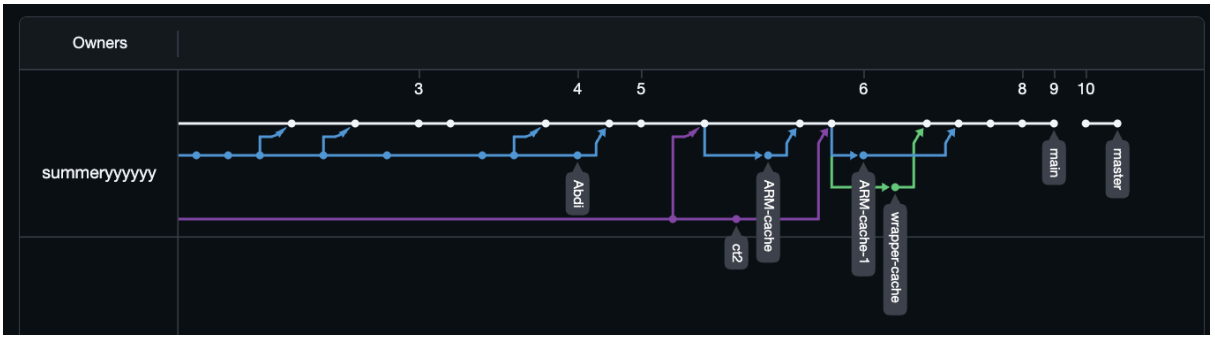

Final result
NEXYS A7
XILINX
DIGILENT
ANALOG DEVICES
ARTIX-7
100T
CSG324
POWER
ON
OFF
BATTERY
PROG UART
SD MICRO
USB HOST
ETHERNET
LINK/ACT
BUSY
CPU RESET
PROG
TEMP SENSOR
AUDIO OUT
MONO

git协作

代码库:

https://github.com/summeryyyyyy/Autumn-SME309





Contributions:

25% 谢毅

25% 郝宇鑫

25% abdi

25% 陈韬